

1. Problemi decisionali e problemi di ottimizzazione

- Qual è la differenza tra problema decisionale e di ottimizzazione?

Un **problema di ottimizzazione** chiede di trovare una soluzione $x \in F$ che **minimizza o massimizza** una funzione obiettivo $c(x)$

In un problema di ottimizzazione, la **formulazione matematica** include:

- **Funzione obiettivo:** es. $\min c(x)$ o $\max c(x)$, rappresenta il costo o guadagno totale
- **Vincoli:** espressi come equazioni o disuguaglianze (lineari o non), limitano l'insieme delle **soluzioni ammissibili** F

💡 Esempio: nella pianificazione di produzione, la funzione obiettivo può essere il ricavo totale $c(x) = 500x_P + 200x_C$, e i vincoli impongono limiti su risorse, quantità e capacità di produzione

Un **problema decisionale** chiede se esiste almeno **una soluzione ammissibile** che soddisfi certe condizioni, es. "esiste $x \in F$ tale che $c(x) \leq k$?".

🔄 È possibile trasformare un problema in **forma decisionale** in un problema di **ottimizzazione** e viceversa, ma **in pratica** il problema decisionale è spesso più semplice da risolvere

- Esempi di modelli

1. 🔄 **Problema di equipartizione:** dividere un insieme di numeri in due sottoinsiemi con somma più simile possibile
2. 🔄 **Circle packing:** posizionare n cerchi dentro un quadrato senza sovrapposizioni massimizzando il raggio
3. 🔄 **Pianificazione produzione (Pintel):** determinare quante unità produrre per massimizzare ricavi

4.  **Problema della fonderia:** minimizzare il costo dei materiali per rispettare specifiche tecniche
5.  **Problema dello zaino:** selezionare oggetti per massimizzare valore sotto un vincolo di capacità

- **Formulazione vincoli e funzione obiettivo**

In un problema di ottimizzazione, la **formulazione matematica** include:

- **Funzione obiettivo:** es. $\min c(x)$ o $\max c(x)$, rappresenta il costo o guadagno totale
- **Vincoli:** espressi come equazioni o disuguaglianze (lineari o non), limitano l'insieme delle **soluzioni ammissibili** F

 Esempio: nella pianificazione di produzione, la funzione obiettivo può essere il ricavo totale $c(x)=500x_P+200x_C$, e i vincoli impongono limiti su risorse, quantità e capacità di produzione

- **Tipi di variabili: continue, discrete, logiche, binarie**

- **Variabili continue:** assumono valori reali (es. quantità di beni)
- **Variabili intere/discrete:** solo valori interi (es. numero di oggetti)
- **Variabili logiche/binarie:** assumono solo i valori 0 o 1, per modellare decisioni tipo “sì/no”

 I problemi con variabili intere o binarie rientrano nella **Programmazione Lineare Intera (PLI)** e sono **computazionalmente più difficili** dei problemi lineari con variabili continue.

- **Classificazione dei problemi**

I problemi si dividono per **difficoltà computazionale**:

- **Problemi polinomiali (classe P):** risolvibili in tempo polinomiale
- **Problemi NP-difficili (NP-hard):** problemi per cui non si conoscono algoritmi efficienti, spesso legati a variabili intere o combinatorie (es. zaino, TSP)

 Alcuni problemi combinatori hanno **molte soluzioni ammissibili** (esponenziali), ma sono comunque trattabili con **rilassamenti, euristiche, o formulazioni alternative**.

2. Grafi e Reti di flusso

Cammini minimi

- **Cos'è il problema dei cammini minimi?**

Sia $G = (N, A)$ un grafo orientato e pesato dove ad ogni arco $(i, j) \in A$ è associato un costo $c_{ij} \in \mathbb{R}$. Per ogni cammino P in G , il costo $C(P) = \sum_{(i,j) \in P} c_{ij}$ è dato dalla somma dei costi degli archi che lo costituiscono. Dati due nodi r e t , definiamo $\mathcal{P}_{rt}$ l'insieme dei cammini (orientati) che connettono r a t . Il *problema del cammino minimo da r a t* ((SP), da *Shortest Path problem*) è quindi

$$(SP) \quad \min \{ C(P) : P \in \mathcal{P}_{rt} \} . \quad (2.5)$$

È il problema di trovare il percorso più economico (cioè con **costo minimo**) tra due nodi di un grafo, dove ogni arco ha un certo costo.

- **Algoritmo di Dijkstra: complessità e implementazione**

```
pseudo
procedure (p, d) = Dijkstra(G, c, r) {
  foreach (i ∈ N) do {
    d[i] = ∞      // Etichetta iniziale: costo infinito
    p[i] = null   // Predecessore sconosciuto
  }
  d[r] = 0       // La radice ha costo 0

  Q = priority_queue() // Coda di priorità (min-heap)
  Q.insert(r, 0)       // Inserisci la radice con priorità 0

  while (Q non è vuota) {
    i = Q.extract_min() // Estrai il nodo con etichetta minima

    foreach ((i, j) ∈ FS(i)) do { // Per ogni vicino j di i
      if (d[i] + c[i][j] < d[j]) { // Se nuovo cammino è più corto
        d[j] = d[i] + c[i][j]      // Aggiorna etichetta
        p[j] = i                  // Aggiorna predecessore
        Q.insert_or_update(j, d[j]) // Inserisci o aggiorna j nella coda
      }
    }
  }
}
```

1. Inizializzazione:

- Si assegna una distanza di 0 al nodo di partenza (nodo sorgente) e una distanza infinita a tutti gli altri nodi.
- Si crea una coda di priorità, che conterrà i nodi da visitare, ordinati per distanza dal nodo sorgente. Il nodo sorgente viene inserito per primo nella coda.

2. Iterazione:

- Si estrae dalla coda il nodo con la distanza minima corrente (il "nodo corrente").
- Per ogni vicino del nodo corrente, si calcola la distanza dal nodo sorgente passando attraverso il nodo corrente (sommando la distanza del nodo corrente all'arco che collega i due nodi).
- Se questa nuova distanza è minore della distanza attuale del vicino, si aggiorna la distanza del vicino e si inserisce il vicino nella coda di priorità (se non è già presente).
- Il nodo corrente viene marcato come "visitato" e non verrà più considerato per futuri aggiornamenti.

3. Ripetizione:

- Il processo di estrazione, calcolo delle distanze e aggiornamento viene ripetuto finché la coda di priorità non è vuota o finché non si raggiunge il nodo di destinazione desiderato

L'algoritmo risultante è noto come *algoritmo di Dijkstra*: è facile verificare che, su grafi con costi non negativi, questo algoritmo ha complessità $O(n^2)$. Infatti, dal Teorema 2.3 discende che non verranno estratti più di n nodi da Q : ad ogni estrazione del nodo di etichetta minima si scandisce l'intera lista in tempo $O(n)$ e quindi il costo totale delle operazioni di gestione della lista è $O(n^2)$. Siccome ogni nodo viene estratto da Q al più una volta, ogni arco (i, j) viene esaminato, una sola volta, se e quando i viene estratto da Q . Le operazioni sui singoli archi sono effettuate in tempo costante, e quindi costano complessivamente $O(m)$; dato che $m < n^2$, la complessità in tempo dell'algoritmo è $O(n^2)$.

L'algoritmo funziona solo se non ci sono costi negativi nel grafo.

• Dimostrazione Bellman: perché cicli negativi invalidano l'albero cammini minimi?

Se nel grafo esistesse un ciclo orientato di costo negativo, allora non esisterebbe (almeno per alcuni nodi del grafo) nessun limite inferiore al costo dei cammini minimi, e quindi non potrebbe esistere nessun vettore di etichette che rispetti le condizioni di Bellman. Infatti, sia $C = \{j_1, j_2, \dots, j_k, j_1\}$ un ciclo negativo, e supponiamo per assurdo che esista un vettore d che rispetta (2.7): si ha

$$\begin{aligned} d_{j_k} &\leq d_{j_{k-1}} + c_{j_{k-1}j_k} \\ d_{j_{k-1}} &\leq d_{j_{k-2}} + c_{j_{k-2}j_{k-1}} \\ &\vdots \\ d_{j_2} &\leq d_{j_1} + c_{j_1j_2} \\ d_{j_1} &\leq d_{j_k} + c_{j_kj_1} \end{aligned}$$

da cui, sommando membro a membro, si ottiene la contraddizione $0 \leq C(C) < 0$.

Se c'è un ciclo negativo (cioè un percorso chiuso con costo totale < 0), puoi continuare a girare in quel ciclo e ridurre il costo all'infinito.

Quindi non esiste un cammino minimo ben definito, perché puoi sempre trovarne uno più economico ripassando nel ciclo.

- **Algoritmo generico per risolvere albero dei cammini minimi**

2.2.3 L'algoritmo *SPT*

Le condizioni di ottimalità presentate nel precedente paragrafo suggeriscono in modo naturale il seguente algoritmo per la determinazione di un albero dei cammini minimi:

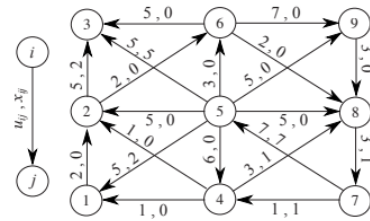
- mediante una visita del grafo dei nodi del grafo, $O(n)$, si determina un albero di copertura radicato in r ed orientato, rappresentato dal vettore $p[\cdot]$ dei predecessori dei nodi, e le relative etichette d_i , che rappresentano il costo dell'unico cammino dell'albero da r a i ;
- si fa partire un processo iterativo che:
 - mediante una visita degli archi del grafo, $O(m)$, controlla se esiste un arco $(i, j) \in A$ tale che $d_i + c_{ij} < d_j$, se ciò non accade l'algoritmo termina in avendo ottenuto un albero ottimo, contenuto in $p[\cdot]$, insieme ad un *certificato* (le etichette che verificano le condizioni di Bellman) che ne dimostra in modo incontrovertibile l'ottimalità;
 - se invece si determina in arco (i, j) che viola le condizioni di Bellman si modifica l'albero sostituendolo a $(p[j], j)$ (ossia ponendo $p[j] = i$), si calcola il vettore delle etichette corrispondente al nuovo albero e si itera; è facile vedere che il ricalcolo delle etichette va effettuato solo per i nodi nel sottoalbero di radice j , in cui tutte le etichette diminuiscono della stessa quantità $d_j - d_i - c_{ij} > 0$, e questo sarebbe in principio $O(n)$ se non fosse per il fatto che la funzione predecessore $p[\cdot]$ non permette di determinare efficientemente i soli nodi nella stella uscente che appartengono all'albero (senza guardarli tutti), il che rende anche questa operazione $O(m)$ al caso pessimo.

L'algoritmo SPT (Shortest Path Tree) costruisce l'albero dei cammini minimi:

1. Inizializza etichette con ∞ , tranne la radice (0).
2. Usa una coda Q con i nodi da esplorare.
3. Ad ogni passo:
 - Esamina gli archi uscenti del nodo estratto.
 - Se trova un arco che migliora il cammino (violazione di Bellman), aggiorna l'etichetta e il predecessore del nodo

L'algoritmo funziona anche con costi negativi nell'albero ma non ci devono essere cicli Negativi

Flusso massimo →



• Cos'è il flusso?

In una rete (grafo), ogni **arco (i, j)** rappresenta un **canale** in cui può "passare qualcosa" (es. acqua, dati, merci...).

☞ Il **flusso $x(i, j)$** è la **quantità effettiva** che stai facendo passare in quell'arco.

- Se $x(i, j) = 5$, significa che stai mandando 5 unità da i a j.

□ Il flusso deve:

- rispettare le **capacità** degli archi: $0 \leq x(i, j) \leq u(i, j)$
- conservare la quantità nei nodi intermedi: **quello che entra = quello che esce**

• Cos'è il valore residuo di un arco?

La **capacità residua** di un arco (i,j) indica **quanta quantità puoi ancora inviare** lungo quell'arco.

Si calcola così:

$$\text{capacità residua} = u(i,j) - x(i,j)$$

Dove:

- $u(i,j)$ è la **capacità massima** dell'arco (quanto può portare al massimo)
- $x(i,j)$ è il **flusso attuale** che già stai mandando lungo l'arco

• Definizione problema del flusso massimo

Dato il grafo orientato $G = (N, A)$, il vettore $u = [u_{ij}]_{(i,j) \in A}$ di capacità superiori degli archi, e due nodi distinti s e t , detti rispettivamente *origine* (o *sorgente*) e *destinazione* (o *pozzo*), il *problema del flusso massimo* ((MF), da Max Flow problem) consiste nel determinare la massima quantità di flusso che è possibile inviare da s a t attraverso G . Più precisamente, si vuole determinare il massimo valore v per cui ponendo $b_s = -v$, $b_t = v$ e $b_i = 0$ per ogni $i \notin \{s, t\}$ esiste un flusso ammissibile x . Questo significa massimizzare il *valore v del flusso x* che soddisfa

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = \begin{cases} -v & i = s \\ v & i = t \\ 0 & \text{altrimenti} \end{cases} \quad i \in N, \quad (2.9)$$

Il problema del flusso massimo consiste nel trovare la massima quantità di flusso che può essere inviata da un nodo sorgente s a un nodo pozzo t in una rete orientata, senza superare le capacità degli archi e rispettando la conservazione del flusso ai nodi intermedi

- **Cosa sono i cammini aumentanti? (flusso massimo)**

$$\theta(P, x) = \min \left\{ \min \{ u_{ij} - x_{ij} : (i, j) \in P^+ \}, \min \{ x_{ij} : (i, j) \in P^- \} \right\} (\geq 0), \quad (2.11)$$

detta *capacità del cammino P rispetto al flusso x* , rappresenta la massima quantità di flusso che, aggiunta agli archi concordi di P , non produce flussi maggiori delle rispettive capacità, e sottratta agli archi discordi di P , non produce flussi negativi. Si noti che può essere $\theta(P, x) = 0$: ciò accade se e solo se almeno uno degli archi concordi è *saturo* oppure almeno uno degli archi discordi è *vuoto*. Se invece $\theta(P, x) > 0$, P è detto un *cammino aumentante*, cioè è un cammino lungo il quale può essere inviata una quantità positiva di flusso da s verso t .

La formula sopra rappresenta una quantità, detta **Capacità del Cammino P rispetto al flusso x** , e una quantità che prende il minimo valore tra:

- Il minimo valore $\{ U_{ij} - X_{ij} \}$ tra gli archi concordi
- Il minimo valore $\{ X_{ij} \}$ tra gli archi discordi

Questa quantità è zero se e presente un arco concorde saturo o un arco discorde con $X_{ij} = 0$, nel caso in cui questa quantità sia > 0 , P viene detto **CAMMINO AUMENTANTE**.

Un **cammino aumentante** è un **cammino da s a t** in cui c'è **capacità residua positiva** su ogni arco.

Aumentare il flusso lungo un cammino aumentante significa **modificare il flusso esistente**, sfruttando la capacità residua degli archi per **spostare sbilanciamenti** tra nodi.

- "Aumentare il flusso" significa:

👉 **modificare il valore del flusso** lungo uno o più archi di un grafo per:

- **portare più "quantità"** (di merce, dati, liquido, ecc.) da un punto a un altro,
- **rispettare i vincoli di capacità e bilancio**,
- **e/o ridurre il costo totale del trasporto**.

Immagina una **rete di tubi** tra serbatoi (nodi):

- Un tubo tra A e B può trasportare fino a 10 litri al secondo.
- Attualmente sta passando solo 3 litri.
- Se **"aumenti il flusso"** su quell'arco, ad esempio da 3 a 5 litri, stai **spingendo più acqua da A a B**.

- **Cos'è un taglio in una rete di flusso?**

Un taglio in una rete è un modo per separare il nodo sorgente s dal nodo pozzo t .

Come si fa?

- Si divide l'insieme dei nodi in due gruppi:
 - Un gruppo contiene s (la sorgente)
 - L'altro contiene t (il pozzo)
- Il taglio è formato dagli archi che vanno da un nodo del primo gruppo a un nodo del secondo.

- **Cos'è la capacità di un taglio?**

È la somma delle capacità di tutti gli archi che vanno dal gruppo di s al gruppo di t .

$$\text{capacità del taglio} = \sum_{(i,j) \in (S,T)} u(i,j)$$

Dove:

- S è il gruppo con s
- T è il gruppo con t
- $u(i,j)$ è la capacità dell'arco che attraversa il taglio

- **Cos'è il taglio minimo?**

È il taglio con la capacità più bassa possibile tra tutti quelli che separano s da t .

- **Algoritmo dei cammini aumentanti per il flusso massimo**

Le proprietà enunciate nel precedente paragrafo permettono di progettare un algoritmo per la determinazione di un flusso massimo: partendo da un flusso nullo, si determina ad ogni passo un cammino aumentante, incrementando il valore del flusso, e ci si ferma quando non esiste più alcun cammino aumentante, restituendo quindi anche un taglio di capacità minima.

```

procedure (  $x, p$  ) = Cammini-Aumentanti(  $G, u, s, t$  ) {
  for(  $x = 0$  ; ; ) {
    (  $p, \theta$  ) = Trova-Cammino(  $G, s, t, x, u$  );
    if(  $p[t] == 0$  ) then break;
     $x$  = Aumenta-Flusso(  $x, p, s, t, \theta$  );
  }
}

```

Procedura 2.6: Algoritmo basato su cammini aumentanti

La procedura *Trova-Cammino* cerca di determinare un cammino aumentante da s a t , dato il flusso x , visitando il grafo residuo G_x e fornendo il corrispondente vettore $p[\cdot]$ dei predecessori dei nodi nell'albero della visita e la capacità $\theta = \theta(P, x)$ del cammino aumentante P individuato tra s e t . Se t non viene raggiunto θ non è definito (ad esempio può avere valore 0), ma comunque l'algoritmo termina: in questo caso il vettore $p[\cdot]$ fornisce un taglio (N_s, N_t) di capacità minima (N_t sono tutti i nodi con predecessore 0, N_s gli altri). Altrimenti si aggiorna il flusso x mediante la procedura *Aumenta-Flusso* che invia lungo il cammino P la quantità di flusso θ , ossia implementa l'operazione di composizione $x = x \oplus \theta P$.

Spiegazione semplice (chiamato anche Ford-Fulkerson):

1. Inizia con flusso = 0.
2. Finché esiste un cammino aumentante da s a t :
 - Trova il minimo valore residuo sugli archi del cammino.

$$\text{capacità residua} = u(i,j) - x(i,j)$$

- Aumenta il flusso su quel cammino di quella quantità.

Per ogni arco del cammino, **aumenti il flusso** di quella quantità .

Questo vuol dire **aggiungere**

$$\theta(P, x) = \min \{ \min \{ u_{ij} - x_{ij} : (i, j) \in P^+ \}, \min \{ x_{ij} : (i, j) \in P^- \} \} \quad (\geq 0)$$

per ogni arco (i, j) del cammino. Se l'arco è al contrario (cioè nella direzione opposta al flusso attuale), **diminuisci** il flusso (cioè "togli" il flusso già mandato per ricalibrarlo)

- Aggiorna la rete residua.

Riduci la capacità residua **sugli archi dove hai aumentato il flusso**

Aggiungi capacità all'indietro **sugli archi dove potresti voler "togliere" flusso**

- **Teorema max-flow min-cut**

Teorema 2.5. (Flusso Massimo-Taglio Minimo) Il massimo valore dei flussi ammissibili su G è uguale alla minima delle capacità dei tagli di G che separano s da t .

Teorema 2.4. Per ogni flusso ammissibile x di valore v e per ogni taglio (N_s, N_t) vale

$$v = x(N_s, N_t) \leq u(N_s, N_t) .$$

Il Teorema 2.4 mostra che, comunque si prenda un taglio che separa t da s , il valore del flusso massimo non può eccedere la capacità di tale taglio. Di conseguenza, possiamo dimostrare che x è un flusso massimo se determiniamo un taglio (N_s, N_t) la cui capacità sia uguale a v . Un taglio di questo tipo è in effetti determinato dalla visita del grafo residuo G_x nel caso in cui t non viene raggiunto. Sia infatti N_s l'insieme dei nodi visitati a partire da s , e $N_t = N \setminus N_s$: tutti gli archi di $A^+(N_s, N_t)$ sono saturi, altrimenti l'algoritmo avrebbe potuto visitare un ulteriore nodo, e analogamente, tutti gli archi di $A^-(N_s, N_t)$ sono vuoti. Dal Teorema 2.4 si ha allora $v = x(N_s, N_t) = u(N_s, N_t)$, ossia il flusso e la capacità del taglio coincidono: di conseguenza, x è un flusso massimo. Un'ulteriore importante conseguenza di questa relazione è che (N_s, N_t) è un *taglio di capacità minima* tra tutti i tagli del grafo che separano s da t .

Teorema max-flow min-cut (Max-Flow = Min-Cut)

Dice che:

Il massimo flusso da s a t in una rete = la capacità del taglio minimo che separa s da t .

Tradotto:

Se blocchi i collegamenti minimi tra s e t (cioè un "taglio"), la somma delle capacità di quegli archi è uguale al massimo flusso che puoi mandare da s a t .

- **Flusso ammissibile (problema di flusso massimo)**

In questo caso, un **flusso ammissibile** è un flusso che:

- rispetta le capacità:

$$x_{ij} \leq u_{ij}$$

- rispetta la **conservazione del flusso** nei nodi interni:

$$\text{flusso entrante} = \text{flusso uscente}$$

- la quantità totale che **esce da s** è uguale a quella che **entra in t** (il valore del flusso)

- **Esistono cicli aumentanti in un problema di flusso massimo?**

Sì, possono esistere, ma non servono a niente per aumentare il valore del flusso totale da s a t . Un **ciclo** nel grafo residuo **non aumenta il flusso da s a t**, perché **parte e finisce nello stesso punto**. Anche se esiste, non serve per aumentare il valore del flusso massimo. Se mandi flusso in un ciclo chiuso, non stai portando nulla in più da s a t .

- **Algoritmo Edmonds-Karp**

Esempio 2.17. Esecuzione dell'algoritmo di Edmonds & Karp

Un esempio di esecuzione dell'algoritmo di Edmonds & Karp è mostrato in Figura 2.16. Per ogni iterazione sono mostrati lo stato iniziale del flusso ed il cammino aumentante selezionato (archi in grassetto); nell'ultima iterazione è mostrato il taglio di capacità minima determinato dall'algoritmo.

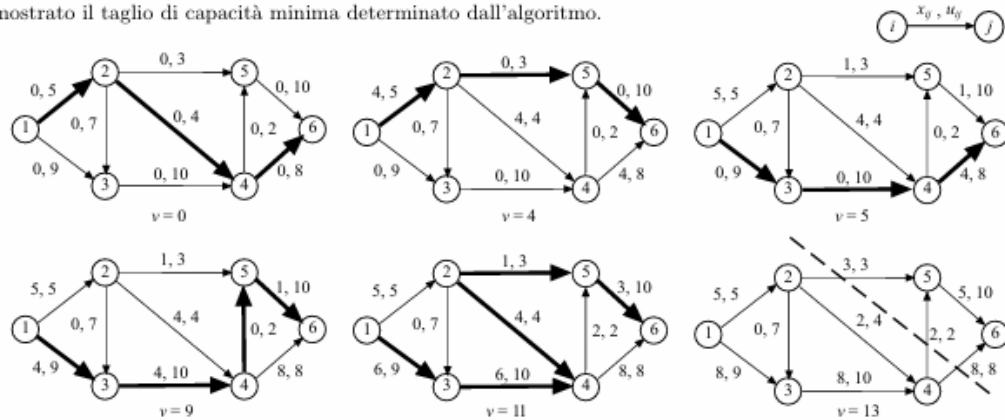


Figura 2.16: Esecuzione dell'algoritmo di Edmonds & Karp

Per l'algoritmo di Edmonds & Karp è possibile dimostrare una valutazione della complessità migliore di quella generale, di cui omettiamo la dimostrazione:

Teorema 2.7. L'algoritmo di Edmonds & Karp ha complessità $O(m^2n)$.

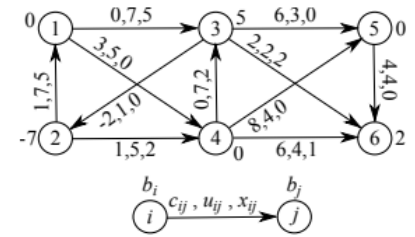
Si noti che questo risultato vale anche nel caso in cui le capacità non siano intere.

1. **Inizializzazione:** Il flusso iniziale su ogni arco è impostato a zero.
2. **Ricerca del percorso di avanzamento:** Si utilizza la BFS per trovare un percorso di avanzamento (shortest path) nel grafo residuo dalla sorgente al pozzo. Il grafo residuo rappresenta la capacità rimanente di ciascun arco nel grafo originale.
3. **Aumento del flusso:** Si determina la capacità minima (residual capacity) lungo il percorso di avanzamento trovato e si incrementa il flusso su tutti gli archi del percorso di quella quantità.
4. **Aggiornamento del grafo residuo:** Si aggiorna il grafo residuo per riflettere il flusso aggiunto.
5. **Ripetizione:** Si ripetono i passi 2-4 finché non esiste più alcun percorso di avanzamento nel grafo residuo.

È più lento ma garantisce la terminazione in tempo polinomiale:

$$O(n \cdot m^2), \text{ dove } n = \text{nodi}, m = \text{archi}.$$

Flusso di costo minimo (Min Cost Flow) ----->



Quali sono i vincoli?

- Conservazione del flusso:** il flusso entrante - flusso uscente = bilancio del nodo

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = b_i \quad i \in N, \quad (2.2)$$

dove $BS(i)$ e $FS(i)$ sono rispettivamente la stella entrante e la stella uscente di $i \in N$ (si veda l'Appendice B); un siffatto x si dice un *flusso* su G , ed il valore x_k (o x_{ij}) è detto *flusso dell'arco* $a_k = (i, j)$.

- Capacità degli archi:** ogni flusso x_{ij} deve essere tra un minimo l_{ij} e un massimo u_{ij}

$$l_{ij} \leq x_{ij} \leq u_{ij} \quad (i, j) \in A. \quad (2.3)$$

ad ogni arco $a_k = (i, j)$ sono associati un *costo* c_k (o c_{ij}), che indica il costo che viene pagato per ogni unità del bene che attraversi l'arco, ed una *capacità inferiore* l_k (l_{ij}) e *superiore* u_k (u_{ij}), che indicano, rispettivamente, il minimo ed il massimo numero di unità di bene che possono attraversare l'arco. In molte applicazioni la capacità inferiore viene assunta uguale a 0, e quindi viene fornita tra i parametri della rete solamente la capacità superiore.

Formalmente:

$$Ex = b$$

$$l \leq x \leq u$$

Cosa è lo sbilanciamento di un nodo?

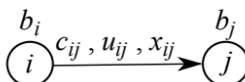
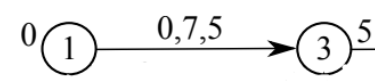
Definiamo Sbilanciamento di un nodo i rispetto ad x la quantità :

$$e_x(i) = \sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} - b_i$$

e indichiamo con $O_x = \{i \in N : e_x(i) > 0\}$ e $D_x = \{i \in N : e_x(i) < 0\}$, rispettivamente, l'insieme dei nodi con *eccedenza di flusso* e con *difetto di flusso*. Si noti come gli sbilanciamenti abbiano "il

Significato dei termini:

- $BS(i)$ l'insieme degli archi entranti nel nodo i (Backwards Star).
- $FS(i)$ l'insieme degli archi uscenti dal nodo i (Forward Star).
- x_{ji} : il flusso che entra nel nodo i proveniente dal nodo j .
- x_{ij} : il flusso che esce dal nodo i verso il nodo j .
- b_i : il valore associato al nodo i , spesso chiamato *sorgente/pozzo/netto di flusso*.

Si ricorda che :  quindi 

- **Quando un flusso è ammissibile ? (Problema di flusso minimo)**

Un **flusso ammissibile** è quindi un **qualsiasi flusso** che:

- rispetta le capacità degli archi:
 - $l_{ij} \leq x_{ij} \leq u_{ij}$ (per ogni arco (i,j) nella rete)

( Questo è il vincolo di capacità sugli archi.)

- rispetta i bilanci b_i ai nodi (offerta/domanda) :
 - **Somma dei flussi in entrata – somma dei flussi in uscita = b_i**

( Questa è la regola di conservazione del flusso, detta anche equazione di bilancio.)

- **Definizione del problema del flusso di costo minimo**

Associando adesso ad ogni flusso x un costo dato dalla somma dei flussi degli archi per il loro costo

$$cx = \sum_{(i,j) \in A} c_{ij} x_{ij}$$

possiamo definire il *problema del flusso di costo minimo* (MCF, da *Min Cost Flow problem*) come

$$(MCF) \quad \min \{ cx : Ex = b, l \leq x \leq u \} \quad (2.4)$$

dove $Ex = b$ rappresenta in forma vettoriale i vincoli di conservazione del flusso (2.2) utilizzando la *matrice di incidenza* del grafo G (si veda il §B.2.1) e $b = [b_i]_{i \in N}$, mentre $l \leq x \leq u$ rappresenta in forma vettoriale i vincoli di capacità (2.3) dove $l = [l_{ij}]_{(i,j) \in A}$ e $u = [u_{ij}]_{(i,j) \in A}$.

È un problema in cui vogliamo trovare **un flusso ammissibile** (cioè che rispetta i vincoli di capacità e conservazione del flusso) **che abbia costo totale minimo**.

L'MCF cerca **un flusso ottimo che sposta le quantità richieste tra nodi** (da quelli che offrono a quelli che domandano) **al costo minimo possibile**, su tutta la rete.

Formalmente:

$$(MCF): \\ \min \{ cx : Ex = b, l \leq x \leq u \}$$

dove:

- **cx** è il costo totale del flusso: somma di tutti i flussi x_{ij} pesati per il costo c_{ij}
- **$Ex = b$** sono i vincoli di conservazione del flusso
- **$l \leq x \leq u$** sono i vincoli di capacità inferiori e superiori sugli archi

- **Qual è la funzione obiettivo del Min Cost Flow?**

La funzione obiettivo del Min Cost Flow è appunto :

$$(MCF) \quad \min \{ cx : Ex = b, l \leq x \leq u \}$$

Con

Associando adesso ad ogni flusso x un costo dato dalla somma dei flussi degli archi per il loro costo

$$cx = \sum_{(i,j) \in A} c_{ij} x_{ij}$$

Vincolo di conservazione del flusso : $Ex = b$

Vincolo di capacità degli archi : $l \leq x \leq u$

- **Cos'è uno pseudoflusso?**

Uno pseudoflusso è un flusso che rispetta i soli vincoli di capacità sugli archi.

Uno pseudoflusso è un flusso che **non necessariamente rispetta i vincoli di conservazione del flusso** (cioè può avere sbilanciamenti ai nodi).

Costo di un flusso /pseudoflusso ammissibile:

$$cx = \sum c_{ij} \cdot x_{ij}$$

Questa formula **vale sempre**, sia che x sia:

- un flusso ammissibile ✓
- uno pseudoflusso ✓

Perché la **funzione obiettivo** dipende solo dai valori di x_{ij} e dai costi c_{ij} assegnati sugli archi.

- **Cos'è un cammino aumentante? (problema di Flusso Minimo)**

Un cammino aumentante è un cammino (cioè una sequenza di archi) che va:

- da un nodo con sbilanciamento positivo (cioè ha più flusso in ingresso che in uscita → sorgente nel flusso residuo),
- a un nodo con sbilanciamento negativo (cioè ha più flusso in uscita che in ingresso → pozzo nel flusso residuo)

Serve per "spostare" flusso lungo il cammino e avvicinarsi a una soluzione ammissibile, cioè che rispetta tutti i bilanci ai nodi

Per ogni cammino P in G , il costo $C(P) = \sum c_{ij}$ è dato dalla somma dei costi degli archi

- **Cos'è un ciclo aumentante? (problema di Flusso Minimo)**

Un ciclo aumentante è un ciclo chiuso nel grafo residuo, cioè un cammino che parte e finisce nello stesso nodo, e ha costo totale negativo.

- Serve per ridurre il costo del flusso, senza cambiare il bilancio nei nodi (perché entra ed esce dallo stesso nodo).
- Aggiungendo flusso su un ciclo aumentante, si abbassa il costo totale del flusso

Per ogni cammino P in G , il costo $C(P) = \sum c_{ij}$ è dato dalla somma dei costi degli archi

Se $C(P) < 0$ il **costo del ciclo è negativo**, e puoi “aumentare flusso” su di esso, **migliori la soluzione** (cioè abbassi il costo totale del flusso).

- **Cos'è uno pseudoflusso minimale? Quando è minimale?**

Definiamo :

minimale uno pseudoflusso x che abbia costo minimo tra tutti gli pseudoflussi aventi lo stesso vettore di sbilanciamento e_x ; si noti che ogni soluzione ottima di (MCF) è un flusso ammissibile minimale, avendo costo minimo tra tutti gli (pseudo)flussi con $e_x = 0$.

Corollario 2.1. Uno pseudoflusso (flusso ammissibile) x è minimale (ottimo) se e solo se non esistono cicli aumentanti rispetto ad x il cui costo sia negativo.

- **Cos'è il grafo residuo?**

Si pone quindi il problema di definire un algoritmo che, dato un grafo G e un flusso ammissibile x , determini (se esiste) un cammino aumentante P rispetto ad x . A questo scopo si introduce il *grafo residuo* $G_x = (N, A_x)$ rispetto al flusso x , dove

$$A_x = A_x^+ \cup A_x^- = \{ (i, j) : (i, j) \in A, x_{ij} < u_{ij} \} \cup \{ (i, j) : (j, i) \in A, x_{ji} > 0 \} .$$

Il grafo residuo, cioè, contiene al più due “rappresentanti” di ciascun arco (i, j) del grafo originale: uno, orientato come l'arco originario, se (i, j) non è saturo e quindi può appartenere all'insieme degli archi *concordi* di un cammino aumentante, mentre l'altro, orientato in modo opposto, se (i, j) non è vuoto, e quindi può appartenere all'insieme degli archi *discordi* di un cammino aumentante. È immediato verificare che G_x permette di ricondurre il concetto di cicli e cammini aumentanti al più usuale concetto di cicli e cammini orientati:

Lemma 2.3. Per ogni cammino aumentante da s a t rispetto ad x in G esiste uno ed un solo cammino orientato da s a t in G_x .

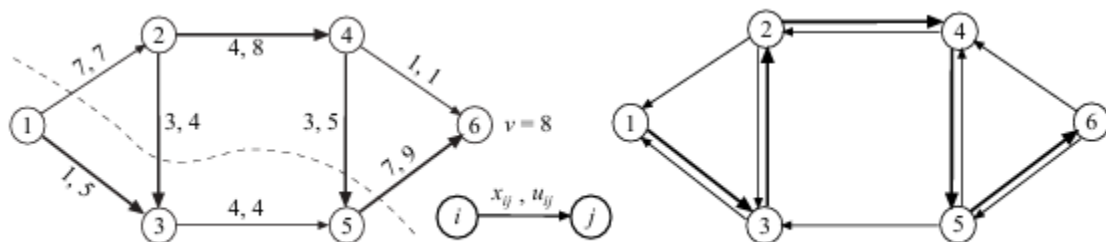


Figura 2.15: un flusso ammissibile x di valore $v = 8$ ed il corrispondente grafo residuo

Un cammino aumentante, se esiste, può quindi essere determinato mediante una visita del grafo residuo G_x a partire da s . Se la visita raggiunge t , allora si è determinato un cammino aumentante (si noti che la visita può essere interrotta non appena questo accada), ed il flusso x non è ottimo perché il cammino aumentante permette di ottenere un nuovo flusso x' di valore strettamente maggiore. Se invece al termine della visita non si è visitato t , allora x è un flusso massimo.

- **Algoritmo della cancellazione di cicli: descrizione e condizioni**

Il Corollario 2.1 suggerisce immediatamente un approccio risolutivo per (MCF): si determina un flusso ammissibile, e poi si utilizzano cicli aumentanti di costo negativo per ottenere flussi ammissibili di costo inferiore. L'algoritmo termina quando non esistono più cicli aumentanti di costo negativo: il flusso ammissibile così determinato è sicuramente di costo minimo. Quando si satura un ciclo aumentante C , inviando lungo i suoi archi un flusso pari a $\theta(C, x)$, si dice che si "cancella" il ciclo C in quanto esso non risulta più aumentante per il flusso $x(\theta)$; non si può però escludere che C possa tornare ad essere aumentante per flussi generati successivamente. Ciò porta alla definizione del seguente algoritmo.

```

procedure  $(x, caso) = \text{Cancella-Cicli}(G, c, b, u)$  {
   $(x, caso) = \text{Flusso-Ammissibile}(G, b, u)$ ;
  if  $(caso == \text{"vuoto"})$  then return;
  for ( ; ) {
     $(C, \theta) = \text{Trova-Ciclo}(G, c, u, x)$ ;
    if  $(\theta == 0)$  then break;
     $x = \text{Cambia-Flusso}(x, C, \theta)$ ;
  }
}

```

Procedura 2.7: Algoritmo basato sulla cancellazione di cicli

La procedura *Flusso-Ammissibile* determina, se esiste, un flusso ammissibile: in tal caso restituisce $caso = \text{"ottimo"}$ ed il flusso x , altrimenti restituisce $caso = \text{"vuoto"}$. Una possibile implementazione di questa procedura è stata discussa nel §2.4.3. La procedura *Trova-Ciclo* determina, dato x , se esiste un ciclo aumentante rispetto ad x di costo negativo: in questo caso restituisce il ciclo individuato C , con il suo verso e la sua capacità $\theta = \theta(C, x)$, altrimenti restituisce $\theta = 0$. Dal Lemma 2.4 segue che il problema di determinare un ciclo aumentante di costo negativo in G rispetto ad x è equivalente al problema di determinare un ciclo orientato di costo negativo in G_x ; tale problema può essere risolto in diversi modi, ad esempio utilizzando la procedura *SPT.L.queue*. In particolare, si può utilizzare la procedura per il problema dell'albero dei cammini minimi con radici multiple (si veda il §2.2.7) con insieme di radici $R = N$; ciò corrisponde ad aggiungere al grafo residuo una radice fittizia r e un arco (r, j) di costo $c_{rj} = 0$ per ogni nodo $j \in N$. Una volta determinato il ciclo C ed il valore θ , la procedura *Cambia-Flusso* costruisce il nuovo flusso $x(\theta) = x \oplus \theta C$.

Trova un flusso ammissibile e **cancella iterativamente i cicli di costo negativo**, migliorando la soluzione finché non ce ne sono più.

Come funziona passo-passo:

1. Parti da un flusso ammissibile (cioè che rispetta i bilanci e le capacità degli archi).
2. Costruisci il grafo residuo del flusso attuale.
3. Cerca un ciclo orientato nel grafo residuo che abbia costo negativo.
4. Se esiste:
 - Calcola quanta quantità puoi far passare su quel ciclo (minima capacità residua).
 - Aumenta il flusso lungo il ciclo di quella quantità.
 - Aggiorna il flusso e il grafo residuo.
5. Ripeti i passaggi 2–4 finché non ci sono più cicli a costo negativo.

- **Algoritmo dei cammini minimi successivi: descrizione, pseudoflusso iniziale, condizioni**

Un approccio sostanzialmente diverso è quello dell'algoritmo dei *cammini minimi successivi*, che mantiene ad ogni passo uno pseudoflusso *minimale* x e determina un *cammino aumentante di costo minimo* da un nodo $s \in O_x$ a un nodo $t \in D_x$ per diminuire, al minor costo possibile, lo sbilanciamento di x . L'uso di cammini aumentanti di costo minimo permette di conservare la minimalità degli pseudoflussi:

```

procedure (  $x$ , caso ) = Cammini-Minimi-Successivi(  $G$ ,  $c$ ,  $b$ ,  $u$  ) {
   $x$  = Inizializza(  $c$ ,  $u$  ); caso = "ottimo";
  while(  $g(x) \neq 0$  ) do
    (  $p$ ,  $d$  ) = Trova-Cammino-Minimo(  $G_x$ ,  $O_x$ ,  $D_x$  );
     $t$  =  $\operatorname{argmin} \{ d[i] : i \in D_x \}$ ;
    if(  $d[t] = \infty$  ) then { caso = "vuoto"; break; }
     $x$  = Aumenta-Flusso(  $x$ ,  $p$ ,  $t$  );
  }
}

```

Procedura 2.8: Algoritmo basato su cammini minimi successivi

La funzione *Inizializza* costruisce e ritorna uno pseudoflusso x minimale: un semplice modo per implementare tale procedura è quello di porre, per ogni $(i, j) \in A$, $x_{ij} = 0$ se $c_{ij} \geq 0$, e $x_{ij} = u_{ij}$ altrimenti. In tal modo i costi degli archi in G_x sono tutti non negativi, e quindi non esistono cicli orientati in G_x (cicli aumentanti rispetto ad x in G) di costo negativo, per cui x è minimale. In effetti è anche facile verificare che lo pseudoflusso così costruito ha costo minimo tra tutti i possibili pseudoflussi per il dato vettore di capacità u . Si noti che questa fase di inizializzazione richiede che non esistano archi con costo negativo e capacità infinita.

La funzione *Trova-Cammino-Minimo* determina un albero dei cammini minimi con insieme di nodi radice O_x ed insieme di destinazioni D_x (si veda il §2.2.7) su G_x ; in altri termini, se $|O_x| > 1$ si aggiunge a G_x un nodo "radice" r collegato a tutti i nodi in O_x con archi a costo nullo, e poi si risolve un problema di albero dei cammini minimi di radice r sul grafo così ottenuto (altrimenti basta usare come radice l'unico nodo in O_x). La procedura termina sicuramente: infatti x è minimale, e quindi non esistono cicli negativi in G_x . Si seleziona quindi il nodo $t \in D_x$ con etichetta $d[t]$ minima, corrispondente al più corto tra tutti i cammini minimi: se $d[t] = \infty$ allora non esiste alcun cammino aumentante da qualsiasi nodo in O_x a qualsiasi nodo in D_x e *Cammini-Minimi-Successivi* termina restituendo caso = "vuoto", in quanto non esiste nessuna soluzione ammissibile per il problema di flusso di costo minimo.

- 1) Parte da uno pseudoflusso minimale.
- 2) Finché ci sono sbilanciamenti (cioè nodi con troppo o troppo poco flusso), cerca un cammino aumentante di costo minimo da un nodo con sbilanciamento positivo (sorgente) a uno con sbilanciamento negativo (destinazione).
- 3) Su questo cammino manda quanto più flusso possibile senza violare capacità o rendere il flusso non ammissibile.
- 4) Ripete il procedimento finché tutti i nodi sono bilanciati.

Ad ogni iterazione trovi il **cammino a costo minimo** (cioè più economico) che ti permette di **aggiustare lo sbilanciamento** del flusso attuale, e lo fai in modo **successivo**, uno alla volta.

- **Differenze tra ciclo e cammino in MCF: uno è primale, uno duale?**

- Un **cammino aumentante** è un cammino da un nodo con sbilanciamento positivo (sorgente) a uno con sbilanciamento negativo (pozzo), lungo il quale si può aumentare il flusso. Serve a **modificare il flusso**, cambiando lo sbilanciamento dei nodi estremi.

“Inviare flusso lungo un cammino aumentante modifica solamente lo sbilanciamento dei nodi estremi del cammino”

- Un **ciclo aumentante** è un cammino chiuso (cioè inizia e finisce nello stesso nodo) in cui si può aumentare flusso senza cambiare gli sbilanciamenti, quindi ha effetto **sul costo** del flusso, non sulla sua fattibilità. È legato al **dual** del problema (ottimalità).

“Inviare flusso lungo un ciclo [...] non modifica lo sbilanciamento di alcun nodo”

✂ **Conclusione semplice:**

- Il **cammino** è uno strumento **primale** (serve per rendere ammissibile un flusso),
- Il **ciclo** è uno strumento **duale** (serve per migliorare il costo del flusso già ammissibile).

- **Come è implementata la funzione Inizializza(G,b,u)?**

Vedi Riposta Cammini-Minimi-Successivi :

```

procedure (  $x$ , caso ) = Cammini-Minimi-Successivi(  $G$ ,  $c$ ,  $b$ ,  $u$  ) {
   $x$  = Inizializza(  $c$ ,  $u$  ); caso = “ottimo”;
  while(  $g(x) \neq 0$  ) do
    (  $p$ ,  $d$  ) = Trova-Cammino-Minimo(  $G_x$ ,  $O_x$ ,  $D_x$  );
     $t$  =  $\operatorname{argmin} \{ d[i] : i \in D_x \}$ ;
    if(  $d[t] == \infty$  ) then { caso = “vuoto”; break; }
     $x$  = Aumenta-Flusso(  $x$ ,  $p$ ,  $t$  );
  }
}

```

Procedura 2.8: *Algoritmo basato su cammini minimi successivi*

- **Relazione tra MCF e PL: analogie e struttura algoritmica generale**

3. Programmazione Lineare

Geometria e struttura della PL

- **Cos'è un vettore?**

Un **vettore** è semplicemente una **lista ordinata di numeri**.

Un vettore $v=(x,y)$ rappresenta il **segmento orientato** che **parte dall'origine** (0,0) e arriva nel punto (x,y).

✈ **Esempio:**

- $v=(2,1) \rightarrow$ vettore che parte da (0,0) e va verso (2,1).

✈ **Esempio:**

Un vettore $x \in \mathbb{R}^3$ è:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Oppure, scritto in orizzontale:

$$x = [x_1, x_2, x_3]$$

Ogni numero (es. x_1, x_2, x_3) si chiama **componente** del vettore. Il numero di componenti si chiama **dimensione del vettore**.

👉 In **PL**, un vettore può rappresentare:

- le **variabili** decisionali (x),
- i **coefficienti** della funzione obiettivo (c),
- i **termini noti** dei vincoli (b).

- **Cos'è una matrice ?**

Una **matrice** è una **tabella di numeri organizzata in righe e colonne**.

✈ **Esempio:**

Una matrice $A \in \mathbb{R}^{3 \times 2}$ (3 righe, 2 colonne) è:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Ogni numero è un **elemento** della matrice, identificato da:

- **riga** (in alto–basso),
- **colonna** (sinistra–destra).

👉 In **PL**, la matrice **A** rappresenta i **coefficienti dei vincoli**.

Ogni **riga** di A è un vincolo, ogni **colonna** è una variabile.

- **Cos'è un vincolo?**

In un problema di Programmazione Lineare (PL), un **vincolo** è una condizione che le **variabili** devono rispettare. I vincoli servono a **limitare le soluzioni possibili**.

Esempio:

“La somma dei prodotti non può superare 100” → è un vincolo:

$$x_1 + x_2 \leq 100$$

- **Cos'è b ?**

È una lista dei valori sul lato destro dei vincoli. Ogni componente di b è il valore che il vincolo non può superare.

Esempio:

$$b = \begin{bmatrix} 10 \\ 15 \\ 7 \end{bmatrix}$$

- **Cos'è Ax ?**

È il risultato numerico che ottieni sostituendo le variabili nei vincoli.

Esempio:

Se

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}$$

Allora:

$$Ax = \begin{bmatrix} 1(1) + 2(2) + 1(3) \\ 3(1) + 0(2) + 2(3) \\ 0(1) + 1(2) + 1(3) \end{bmatrix} = \begin{bmatrix} 8 \\ 9 \\ 5 \end{bmatrix}$$

- Cos'è $Ax \leq b$?

Questa è una **notazione compatta** per rappresentare **tanti vincoli contemporaneamente**.

$$Ax = \begin{bmatrix} 1(1) + 2(2) + 1(3) \\ 3(1) + 0(2) + 2(3) \\ 0(1) + 1(2) + 1(3) \end{bmatrix} = \begin{bmatrix} 8 \\ 9 \\ 5 \end{bmatrix}$$

$$b = \begin{bmatrix} 10 \\ 15 \\ 7 \end{bmatrix}$$

- se $Ax \leq b$, allora le condizioni sono **soddisfatte** $\leq$
- se no, la soluzione è **non ammissibile**.

- Cosa vuol dire “vincoli con lato destro nullo”?

Vuol dire che il vettore b è tutto zero:

$$b = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \Rightarrow Ax \leq 0$$

- Cosa è una direzione?

In geometria (e in Programmazione Lineare), una **direzione** è rappresentata da un **vettore** che descrive in quale “verso” ti stai muovendo nello spazio, **senza indicare un punto preciso di partenza**.

👉 In pratica:

- Una **direzione** è un vettore $d \in \mathbb{R}^n$ che rappresenta un **movimento** nello spazio.
- È come dire: “sto andando **verso destra**, o **verso l’alto**, o **verso il basso a sinistra**...”

💡 **Importante:** Una **direzione** è diversa da un **punto**.

- Un punto è una **posizione** (es: $(2, 3)$).
- Una direzione è un **verso**, cioè **dove potresti andare da lì in poi** (es: $d = (1, 0)$: vai a destra).

- **Cos'è un poliedro?**

Un **poliedro** è l'insieme delle soluzioni di un sistema di disuguaglianze lineari del tipo:

$$P = \{x \in \mathbb{R}^n : Ax \leq b\}$$

- Dove A è una matrice e b un vettore. Si tratta di un **insieme convesso**, perché è l'intersezione di semispazi, che sono convessi. Quindi, se prendi due punti dentro il poliedro, anche il segmento che li collega sta dentro il poliedro.

oppure una sua traslazione, cioè un iperpiano / semispazio affine ($b_i \neq 0$). Un insieme P è un *poliedro* se è esprimibile come intersezione di un numero finito m di semispazi, cioè se esistono una **matrice** $A \in \mathbb{R}^{m \times n}$ ed un vettore $b \in \mathbb{R}^m$ per cui

$$P = \{x : Ax \leq b\}.$$

Se il numero di variabili è pari a due oppure tre, un problema di *PL* può essere descritto (e risolto) mediante una sua rappresentazione geometrica nel piano o nello spazio. In Figura 3.1 ciò è mostrato per il problema della *Pintel*: avendo indicato con x_1 il numero di Pintium e con x_2 il numero di Coloron prodotti, l'area evidenziata col tratteggio rappresenta l'*insieme ammissibile*, o regione ammissibile, del problema, cioè l'insieme di tutti i punti che soddisfano i vincoli. Si tratta dell'intersezione di un certo numero di semipiani, uno per ogni vincolo: un insieme di questo tipo viene detto **poliedro** (*politopo* nel caso particolare in cui sia limitato, come nell'esempio che stiamo considerando). Nel seguito vedremo che tale insieme è convesso; infatti, si usa spesso il termine **poliedro convesso** per caratterizzare l'insieme ammissibile di un problema di *PL*. Nella figura sono evidenziati i vincoli per mezzo delle rette luogo dei punti che li soddisfano come uguaglianze: tali rette costituiscono la frontiera dei semipiani che essi definiscono. I vincoli di non negatività sono individuati dagli assi cartesiani. Le rette corrispondenti ai vincoli individuano nel **poliedro** delle *facce* e dei *vertici*: sono vertici ad esempio i punti $[4, 1]$ e $[1, 7]$, ed è una faccia il segmento che li unisce. Una delle principali proprietà dei poliedri è la loro *convessità*, dove ricordiamo che un insieme C è detto convesso se, comunque presi due punti x ed y appartenenti a C , il segmento avente x ed y per estremi è contenuto in C , cioè $\text{conv}(x, y) = \{z = \alpha x + (1 - \alpha)y : \forall \alpha \in [0, 1]\}$, è formato solamente da punti di C : $\text{conv}(x, y) \subseteq C$. Un semispazio è un insieme convesso, ed essendo l'intersezione di insiemi convessi a sua volta un insieme convesso, anche un poliedro è un insieme convesso.

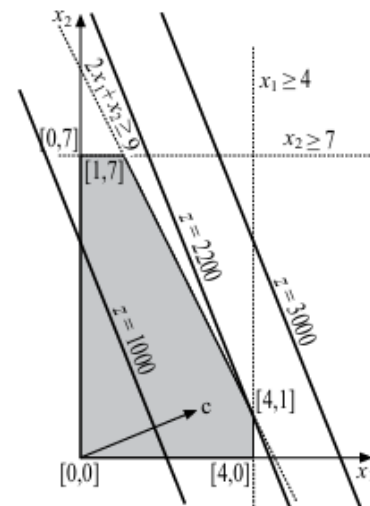


Figura 3.1: Il problema della Pintel

- **Cosa significa "somma pesata" ?**

Una **somma pesata** (detta anche **combinazione lineare con pesi**) è una somma in cui ogni vettore è moltiplicato per un numero (peso) prima di essere sommato.

✚ **Formula generale:**

$$\lambda_1 v_1 + \lambda_2 v_2 + \dots + \lambda_k v_k$$

dove:

- $v_1, v_2, \dots, v_k$ sono **vettori** in $\mathbb{R}^n$,
- $\lambda_1, \lambda_2, \dots, \lambda_k \in \mathbb{R}$ sono i **pesi** (numeri reali, positivi o negativi).

- Cosa significa “vettore chiuso per moltiplicazione positiva” ?

Se un vettore è nell'insieme, allora anche tutte le sue versioni moltiplicate per un numero positivo sono anch'esse nell'insieme.

In simboli:

$$x \in C \text{ e } \lambda \geq 0 \Rightarrow \lambda x \in C$$

✓ **Esempio concreto:**

Prendiamo il vettore:

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Supponiamo che $x \in C$, cioè che il vettore stia nell'insieme C .

Se C è chiuso per moltiplicazione positiva, allora anche i seguenti vettori stanno in C :

- $\lambda = 2 \Rightarrow 2x = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$
- $\lambda = 0.5 \Rightarrow 0.5x = \begin{bmatrix} 1 \\ 1.5 \end{bmatrix}$
- $\lambda = 0 \Rightarrow 0x = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Tutti questi vettori stanno nel cono generato da x .

👉 Questo è ciò che si intende con:

“il cono è chiuso rispetto alla moltiplicazione per scalari positivi”

Chiuso per moltiplicazione positiva” =

“Se un vettore è nell'insieme, anche tutte le sue versioni **allungate o accorciate** (ma nella stessa direzione) sono ancora nell'insieme, purché il fattore sia ≥ 0 .

- Cos'è un cono?

Un **cono** è un insieme di vettori chiuso rispetto alla moltiplicazione per scalari **positivi**. Significa che, se un vettore è nel cono, allora anche tutte le sue versioni “allungate” (cioè moltiplicate per un numero positivo) lo sono.

$$x \in C, \alpha \geq 0 \Rightarrow \alpha x \in C$$

Esempio: il primo quadrante del piano $\mathbb{R}^2$ (cioè $x \geq 0, y \geq 0$) è un cono.

- **Cos'è un cono convesso?**

È un cono che è anche **convesso**, cioè se due vettori stanno nel cono, allora anche **qualsiasi combinazione positiva di essi** (somma pesata con coefficienti ≥ 0) sta nel cono.

$$x, y \in C, \lambda, \mu \geq 0 \Rightarrow \lambda x + \mu y \in C$$

- **Cos'è un cono poliedrico?**

È un cono convesso che è anche definito da un **sistema di disuguaglianze lineari con lato destro nullo**:

$$C = \{x \in \mathbb{R}^n : Ax \leq 0\}$$

Questa forma garantisce che ogni vettore x che soddisfa i vincoli può essere moltiplicato per $\alpha \geq 0$ restando ancora nel cono.

- **Cos'è un cono recessivo?**

Il **cono recessivo** (detto anche **di recessione**) è l'insieme delle **direzioni** lungo cui ci si può muovere **all'infinito** restando all'interno di un poliedro P . Formalmente:

$$\text{rec}(P) = \{d \in \mathbb{R}^n : Ad \leq 0\}.$$

In altri termini, azzerando il lato destro dei vincoli si “cattura il comportamento all'infinito” di un poliedro.

Il **cono di recessione** $\text{rec}(P)$ è l'insieme di **tutte le direzioni di recessione** del poliedro P .

È un cono perché:

- Se $d \in \text{rec}(P)$, allora anche $\lambda d \in \text{rec}(P)$ per $\lambda \geq 0$
- Ed è **convesso**: se due direzioni sono di recessione, anche la loro somma lo è.

🧠 **Intuizione:**

È come dire: "Queste sono **tutte le direzioni** lungo cui posso **fuggire all'infinito dentro il poliedro**".

- **Cos'è un cono finitamente generato?**

È un cono che si può ottenere come **combinazione conica** (cioè somma pesata con coefficienti ≥ 0) di un **insieme finito** di vettori $v_1, \dots, v_t$:

$$\text{cono}(V) = \left\{ \sum_{i=1}^t \nu_i v_i : \nu_i \geq 0 \right\}$$

- **Cos'è una direzione ammissibile?**

Dato $\bar{x} \in P$, un vettore $\xi \in \mathbb{R}^n$ è detto una *direzione ammissibile* per $\bar{x}$ se esiste un $\bar{\lambda} > 0$ per cui $x(\lambda) = \bar{x} + \lambda\xi$ è ammissibile per (P) per ogni $\lambda \in [0, \bar{\lambda}]$, cioè per ogni $i = 1, \dots, m$ vale

$$A_i x(\lambda) = A_i \bar{x} + \lambda A_i \xi \leq b_i \quad . \quad (3.11)$$

Se prendiamo un punto $\bar{x}$ dentro il poliedro P , e troviamo una direzione ξ tale che, spostandoci leggermente lungo di essa, restiamo comunque dentro P , allora ξ è una **direzione ammissibile** per $\bar{x}$.

Questo è vero se, per ogni vincolo, il punto spostato $\bar{x} + \lambda\xi$ **continua a rispettare la disuguaglianza** $A_i x \leq b_i$.

Formalmente:

$$\exists \bar{\lambda} > 0 \text{ tale che } \forall \lambda \in [0, \bar{\lambda}], \quad x(\lambda) = \bar{x} + \lambda\xi \in P$$

Questo è un **punto fisso** nel poliedro P , cioè una **soluzione ammissibile** già data.

Pensa a $\bar{x}$ come al **punto di partenza** da cui analizzi la geometria locale (le direzioni, i vincoli attivi, ecc.).

Esempio:

$$\bar{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

significa: sto guardando **cosa succede nel punto (2, 3)** del mio poliedro.

ξ (si legge: "xi", è una **direzione)**

È un **vettore in $\mathbb{R}^n$** che rappresenta una **direzione in cui ti vuoi muovere** partendo da $\bar{x}$.

Pensa a ξ come a una **freccia** che indica "dove sto andando" rispetto a $\bar{x}$.

Questo è il **vettore ξ moltiplicato per uno scalare $\lambda \geq 0$** .

- λ è un **numero reale positivo** (es: 0.1, 1, 3, ...),
- $\lambda\xi$ è un **vettore nella stessa direzione di ξ** , ma con **lunghezza proporzionale a λ** .

👉 Questo è il modo matematico per dire:

"Mi sposto lungo la direzione ξ , di un certo passo λ ."

Questa è la **posizione del punto dopo lo spostamento**.

Esempio:

Se

- Parti da $\bar{x}$
- Ti muovi lungo la direzione ξ
- Con un "passo" lungo λ

allora:

$$\bar{x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad \xi = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad \lambda = 0.5$$

$$x(\lambda) = \bar{x} + \lambda\xi = \begin{bmatrix} 2 \\ 3 \end{bmatrix} + 0.5 \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 2.5 \end{bmatrix}$$

Quindi $x(\lambda)$ è il **punto raggiunto** dopo lo spostamento.

$$x(\lambda) = \bar{x} + \lambda\xi$$

- **Cos'è una direzione di crescita?**

Una direzione ξ è di crescita se, muovendoci lungo di essa, la **funzione obiettivo aumenta**:

$$c \cdot \xi > 0 \Rightarrow \text{direzione di crescita}$$

La funzione obiettivo in forma vettoriale è:

$$Z = C \cdot X = C_1 X_1 + C_2 X_2 + \dots + C_n X_n$$

- c è il vettore dei coefficienti della funzione obiettivo (cioè il "gradiente")
- ξ è una **direzione nello spazio delle variabili**
- Se fai un passo in direzione ξ , allora z **cambia** di $c \cdot \xi$

Se $c \cdot \xi > 0$, stai andando **verso valori maggiori di z** , quindi ξ è una **direzione di crescita**

Esempio:

Supponiamo di avere la funzione obiettivo:

$$\max z = 3x_1 + 2x_2$$

Allora il vettore $c = [3, 2]$

Ora proviamo con vari vettori ξ e calcoliamo il prodotto scalare $c \cdot \xi$

$$\begin{aligned} \xi &= [-2, 4] \\ \rightarrow c \cdot \xi &= 3 \cdot (-2) + 2 \cdot 4 = -6 + 8 = 2 > 0 \\ \checkmark &\Rightarrow \xi \text{ è direzione di crescita} \end{aligned}$$

- **Qual è l'insieme di tutte le direzioni ammissibili per $\bar{x}$? (cono simpliciale)**

Quando $\bar{x}$ è una **soluzione di base**, questo insieme prende una forma particolare: è un **cono simpliciale**, cioè generato da n vettori linearmente indipendenti.

È il cono delle direzioni ammissibili:

$$C(\bar{x}) = \{d \in \mathbb{R}^n : A_{I(\bar{x})} d \leq 0\}$$

$$\{x : A_B x \leq 0\} = \{x : -A_B x = \nu, \nu \geq 0\} = \{x = -A_B^{-1} \nu : \nu \geq 0\} . \quad (3.12)$$

Questo significa che mentre per la famiglia dei cono poliedrici la conversione in cono finitamente generato, sebbene sempre possibile (cf. Teorema 3.3), può avere un costo esponenziale in quanto il cono finitamente generato può avere un numero di generatori esponenziale, i **coni simpliciali** hanno esattamente n generatori, corrispondenti all'opposto delle colonne della matrice inversa A_B^{-1} , e quindi una loro descrizione si può calcolare con complessità $O(n^3)$ o inferiore.

A	matrice dei coefficienti del sistema $Ax = b$	A_B^{-1}	matrice inversa di A_B
B	insieme degli indici delle variabili di base	ν (nu greca)	vettore con $\nu \geq 0$ (serve per riscrivere una disequazione come uguaglianza)
A_B	sottomatrice di A con le colonne di B (una matrice quadrata)	$x = -A_B^{-1} \nu$	formula esplicita delle soluzioni x che soddisfano $A_B x \leq 0$

- **Cos'è il passo massimo?**

È il massimo valore di λ per cui possiamo spostarci lungo una direzione ξ a partire da $\bar{x}$ senza violare alcun vincolo.

$$\lambda_i = \frac{b_i - A_i \bar{x}}{A_i \xi} \quad (\text{per ogni vincolo con } A_i \xi > 0)$$

Il passo massimo è:

$$\bar{\lambda} = \min_i \lambda_i$$

Significato dei simboli:

- A_i : riga i-esima della matrice dei vincoli A
- $\bar{x}$: punto attuale (soluzione di base)
- ξ : direzione in cui ci si muove
- b_i : termine noto del vincolo i-esimo

- **Quando un passo è ammissibile?**

Definizione:

Un passo è ammissibile se, muovendosi da $\bar{x}$ nella direzione ξ , si rimane dentro la regione ammissibile. Cioè:

$$x(\lambda) = \bar{x} + \lambda \xi \quad \text{deve soddisfare } Ax(\lambda) \leq b$$

- **Cosa significa $\bar{x}$?**

$$\bar{x} = A_B^{-1} b_B$$

È la **soluzione associata a una base B**, cioè il **punto che ottieni risolvendo i vincoli associati a quella base**.

Nel caso standard:

- n : numero di variabili
- m : numero di vincoli

Quando risolvi $A_B \bar{x}_B = b_B$, ottieni una **soluzione di base $\bar{x}$** .

👉 Se questa soluzione:

- È **ammissibile** (cioè $\bar{x} \geq 0$),
- E deriva da n vincoli attivi indipendenti,

allora **geometricamente è un vertice del poliedro ammissibile**.

- **Cos'è una base?**

Una **base** è un sottoinsieme di n vincoli (tra gli m totali) tali che le righe corrispondenti della matrice A formano una matrice quadrata e invertibile A_B .

Serve per calcolare una soluzione di base:

$$\bar{x} = A_B^{-1}b_B$$

Supponiamo di avere il seguente problema:

$$\max z = 3x_1 + 2x_2$$

$$\text{soggetto a: } \begin{cases} x_1 + x_2 \leq 4 \\ 2x_1 + x_2 \leq 5 \\ x_1, x_2 \geq 0 \end{cases}$$

Introduciamo le **variabili di scarto** s_1, s_2 per trasformare le disuguaglianze in uguaglianze:

$$\begin{cases} x_1 + x_2 + s_1 = 4 \\ 2x_1 + x_2 + s_2 = 5 \\ x_1, x_2, s_1, s_2 \geq 0 \end{cases}$$

- Matrice dei vincoli:

$$A = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 2 & 1 & 0 & 1 \end{bmatrix}$$

- Vettore $b = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$

- Variabili: $x = \begin{bmatrix} x_1 \\ x_2 \\ s_1 \\ s_2 \end{bmatrix}$

♦ Ora prendiamo come base $B = \{x_1, x_2\}$

Quindi prendiamo la **sottomatrice**:

$$A_B = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}$$

Verifica se è invertibile:

$$\det(A_B) = (1)(1) - (2)(1) = -1 \neq 0 \Rightarrow \checkmark \text{ è invertibile}$$

✓ Quindi x_1 e x_2 formano una base valida.

Calcolo della soluzione di base:

$$\bar{x}_B = A_B^{-1}b = [\text{calcolo inversa}] \cdot \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

Inversa di:

$$A_B = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \Rightarrow A_B^{-1} = \frac{1}{-1} \begin{bmatrix} 1 & -1 \\ -2 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ 2 & -1 \end{bmatrix}$$

Allora:

$$\bar{x}_B = \begin{bmatrix} -1 & 1 \\ 2 & -1 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -1(4) + 1(5) \\ 2(4) - 1(5) \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

Quindi:

- $x_1 = 1$
- $x_2 = 3$
- Le variabili **non in base** (cioè s_1, s_2) valgono 0

✦ Conclusione

- Base = $\{x_1, x_2\}$
- Soluzione di base = $(1, 3, 0, 0)$
- È anche **ammissibile**, perché tutti i valori sono $\geq 0 \Rightarrow$ è una **base ammissibile** ✓

- **Cos'è una base ammissibile?**

Una **base ammissibile** è una base per cui la **soluzione di base associata** è **ammissibile**, cioè **tutte le variabili** (anche quelle non in base) hanno **valori ≥ 0** .

Più precisamente:

- Una **base** è un insieme di n colonne **linearmente indipendenti** di A .
- Se calcoli la soluzione $\bar{x} = A_B^{-1}b_B$ e **tutti i valori sono ≥ 0** , allora quella base è **ammissibile**.

- **Problema primale**

Un problema nella forma:

$$(P) \quad \max \{ cx : Ax \leq b, x \geq 0 \} \quad (3.13)$$

- **Problema duale**

Un problema nella forma:

$$(D) \quad \min \{ yb : yA \geq c, y \geq 0 \} \quad (3.14)$$

✓ **Cos'è il problema duale?**

Il **problema duale** è un altro problema di programmazione lineare costruito a **partire dal problema primale**, che ha le **stesse informazioni** ma scritte in modo **simmetrico**.

Ogni problema primale ha il suo **problema duale associato**.

Le **variabili** del primale diventano i **vincoli** del duale, e viceversa.

📌 **Scopo del duale:**

- Dare **informazioni economiche** (prezzi ombra, costi minimi)
- Verificare se il **primale ha soluzioni ottime**
- Trovare **valori ottimi equivalenti** al primale

- **Primale geometrico**

Definizione:

È un modo visuale per eseguire l'algoritmo del simplesso, disegnando le rette dei vincoli nel piano (valido solo per problemi con 2-3 variabili). Si cercano i vertici del poliedro e si migliora la funzione obiettivo seguendo le direzioni di crescita.

- **Definizione di rilassamento. La funzione obiettivo cambia?**

Definizione:

Il rilassamento consiste nel **modificare i vincoli** del problema per renderlo più facile da risolvere. Ad esempio:

- togliere il vincolo di interezza (rilassamento lineare di un intero)
- rilassare alcuni vincoli disuguali o rimuoverli

La funzione obiettivo resta la stessa, ma può cambiare l'insieme delle soluzioni ammissibili, quindi il valore ottimo sì.

Citazione (parafrasata):

Il concetto viene spesso usato nel capitolo di problemi interi o combinatori, ma non è descritto con la parola "rilassamento" esplicitamente nel tuo file. Comunque, da quanto detto nei punti precedenti, possiamo affermare:

Ammissibilità, degenerazione e ottimalità

Si inizia a introdurre concetti fondamentali per la risoluzione del problema.

- **PL: se il problema è vuoto come si dimostra?**

Un **problema è vuoto** se **non ha soluzioni ammissibili**, cioè $Ax \leq b$ non ammette soluzioni.

✚ Si dimostra:

- con l'**algoritmo del simplesso**: se **nessuna base ammissibile esiste**
 - oppure usando il **dual**: se il **duale è illimitato inferiormente**, allora il primale è **vuoto**
-
- trovare un **sistema di vincoli che non può essere soddisfatto**
 - oppure usare il **teorema di Farkas** per mostrare che **nessun $x \geq 0$ soddisfa $Ax = b$**

✓ **ESEMPIO – Problema di PL vuoto**

$$\begin{array}{ll}\max & x_1 + x_2 \\ \text{s.t.} & x_1 + x_2 \leq 1 \\ & x_1 + x_2 \geq 3 \\ & x_1, x_2 \geq 0\end{array}$$

Guarda i due vincoli principali:

- $x_1 + x_2 \leq 1$
- $x_1 + x_2 \geq 3$

Ma **non può esistere un punto** che soddisfa **entrambi** allo stesso tempo.

● Nessun (x_1, x_2) può avere somma ≤ 1 e ≥ 3 contemporaneamente.

Quindi:

👉 L'insieme ammissibile è vuoto $\Rightarrow$ nessuna soluzione $\Rightarrow$ problema vuoto

📐 **GEOMETRIA (facile da visualizzare)**

- La retta $x_1 + x_2 = 1$ divide il piano: **tutto sotto** è ammissibile per il primo vincolo
- La retta $x_1 + x_2 = 3$ dice che devi stare **sopra** questa retta

✗ Le due zone **non si sovrappongono** $\rightarrow$ **non c'è intersezione**.

- **Hai un problema di programmazione lineare , se uno ti dice che e sicuramente vuoto , come si dimostra?**
- Per poter dimostrare che il problema in questione è vuoto basterebbe seguire l'algoritmo del simplesso duale associato e calcolare η -b ; se η -B ≤ 0 allora il problema primale è vuoto.
- Questo perché η -b è una direzione ammissibile di decrescita per il problema duale, che verrà estesa a $\mathbb{R}^m$, se η -b è minore o uguale a zero allora vuol dire che ho una direzione di decrescita illimitata risultando in un problema duale inferiormente illimitato. Questo causa il mio problema primale a essere vuoto.
- **Cos'è $\bar{y}$ (cioè "y soprattitolato")?**

È la soluzione del problema duale.

- Come $\bar{x}$ è la soluzione ottima del primale,
- Così $\bar{y}$ è la soluzione ottima del duale

- **Cos'è la “soluzione duale complementare”?**

Quando applichi il simplesso primale, calcoli una base B e da quella ottieni:

- $\bar{x} = A_B^{-1}b_B$: la soluzione primale
- $\bar{y} = c_B A_B^{-1}$: la soluzione duale complementare

✦ Si chiama “complementare” perché è calcolata a partire dalla base B del primale, e ti dice se la soluzione è ottima o no.

- **Quando una base risolve il primale?**

Una base risolve il problema primale (di programmazione lineare) quando, trovando una soluzione ammissibile di base, soddisfa tutte le condizioni del problema primale, ovvero quando la base ammissibile di base porta ad un valore ottimo della funzione obiettivo. In altre parole, una base è una soluzione ottima del problema primale quando, tra tutte le possibili soluzioni ammissibili, quella trovata dalla base offre il miglior valore per la funzione obiettivo, che sia il massimo (nel caso di problema di massimizzazione) o il minimo (nel caso di problema di minimizzazione).

- **Cosa comporta una base primale degenerare?**

Base primale degenerare --> condizione necessaria ma non sufficiente per provare la presenza di più soluzioni nel duale

- **Cosa comporta una base duale degenerare?**

Base duale degenerare --> condizione necessaria ma non sufficiente per provare la presenza di più soluzioni nel primale

- **Base duale ammissibile: definizione e identificazione**

Una **base duale** è detta **ammissibile** se la **soluzione duale associata alla base** soddisfa tutti i vincoli del **problema duale**.

Nel linguaggio del simplesso:

Una base è duale ammissibile se
 $\bar{y} = c_B A_B^{-1}$ è una soluzione ammissibile del duale, cioè

✓ **Come si identifica una base duale ammissibile?**

Segui questi passi:

1. **Scegli una base B** → cioè un sottoinsieme di n colonne linearmente indipendenti della matrice dei vincoli A .
2. Calcola $\bar{y} = c_B A_B^{-1}$ → questa è la **soluzione duale complementare**.
3. Verifica che $\bar{y} A_i \geq c_i$ per ogni colonna A_i (anche quelle **non in base**).
 - Se è vero → **la base è duale ammissibile**
 - Se no → non lo è

- La base è un **sottoinsieme di vincoli scelti** per cercare di risolvere il primale.
- Ma ogni base corrisponde anche a una **soluzione del duale**.
- Se quella soluzione soddisfa i **vincoli del duale**, allora è **duale ammissibile**.

🎯 **Perché ci interessa?**

👉 Perché se troviamo una **base** che è sia:

- **ammissibile** (cioè genera $\bar{x} \geq 0$)
- e **duale ammissibile** (cioè $\bar{y} A_i \geq c_i$)

allora abbiamo trovato la soluzione ottima!

- **Condizione degli scarti complementari: cos'è e perché interessa?**

La condizione degli scarti complementari è una condizione che viene rispettata dalle soluzioni ottime dei problema Primale e Duale. Questa condizione ci permette di individuare se la soluzione di base attuale è una soluzione ottima. In questa situazione i valori della funzione obiettivo del primale e la funzione obiettivo del duale coincidono.

Cioè $cx = yb$;

ma se $c = yA \Rightarrow yAx = yb \Rightarrow 0 = yb - yAx$

$y^T (b - Ax) = 0$

Questa equazione mi dice che se $Ax < b$ (vincolo non attivo) allora $y=0$; Se $y>0$ allora $\Rightarrow Ax = b$ (vincolo attivo).

Questo teorema mi dice che quando si ha un vincolo attivo nel primale allora esiste uno scarto nel duale, ossia esiste la y corrispondente alla riga del vincolo attivo che è maggiore di zero.

Al contrario se ho un vincolo non attivo nel primale, e quindi quel vincolo ha uno scarto, allora la y corrispondente alla riga del vincolo è necessariamente zero.

Potrebbe capitare che sia il vincolo sia la variabile siano uguali a zero.

Questo significa che possono esistere molteplici soluzioni ottime per il primale.



Condizione degli scarti complementari – DEFINIZIONE

È una **condizione di ottimalità** che collega le **soluzioni del primale e del duale**.

Essa deve essere **soddisfatta contemporaneamente** per dire che una **coppia di soluzioni è ottima**.

La formula generale è:

$$y^T (Ax - b) = 0$$

oppure, equivalentemente:

$$(c - A^T y)^T x = 0$$



Significato pratico (intuitivo):

Se una **variabile è positiva** nel primale, allora il **vincolo corrispondente nel duale è attivo**

Se una **variabile è positiva** nel duale, allora il **vincolo corrispondente nel primale è attivo**



Cioè: **o lo scarto è zero, o la variabile è zero. Mai entrambe positive.**

📌 In forma "complementare":

✎ Per ogni variabile x_j del primale:

$$x_j \cdot (c_j - a_j^T y) = 0$$

👉 Se $x_j > 0$, allora $c_j = a_j^T y$

👉 Se $c_j - a_j^T y > 0$, allora $x_j = 0$

✎ Per ogni variabile y_i del duale:

$$y_i \cdot (a_i x - b_i) = 0$$

👉 Se $y_i > 0$, allora il vincolo $a_i x = b_i$ è **attivo**

👉 Se $a_i x < b_i$, allora $y_i = 0$

✅ Perché è importante?

Perché questa condizione ci permette di dire se una **base** è **ottima**, **senza cercare altre soluzioni**.

📌 Viene usata **nel simplesso** per sapere se si può **fermare l'algoritmo**.

- **Soluzioni multiple nel primale <-> duale? Quando sono uniche?**

Caso	Risultato
Primale ha più ottimi	Il duale può averne uno o più
Duale ha più ottimi	Il primale può averne uno o più
Entrambi unici	Se funzione obiettivo "tocca" solo un vertice
Entrambi multipli	Se ottimo è su una faccia e duale ha molte y

✅ 1. Quando il primale ha soluzioni multiple?

✎ Esempio:

$$\begin{aligned} \max \quad & x_1 + x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 4 \\ & x_1, x_2 \geq 0 \end{aligned}$$

👉 Tutti i punti (x_1, x_2) su $x_1 + x_2 = 4$ e $x_1, x_2 \geq 0$ sono **soluzioni ottime**

👉 Perché danno tutti: $x_1 + x_2 = 4$

🟢 Infiniti ottimi $\Rightarrow$ soluzione non unica

✅ 2. Quando la soluzione è unica?

La soluzione è unica se:

- La retta della funzione obiettivo **tocca la regione ammissibile in un solo punto**
- L'ottimo **non è su un segmento** o su una faccia del poliedro, ma in un **solo vertice**

📌 Geometricamente:

Se l'ottimo è un **vertice solo** $\Rightarrow$ **soluzione unica**

Se l'ottimo è su un **lato** o **faccia** $\Rightarrow$ **soluzioni multiple**

Dualità

- **Scrivi il duale dato il primale**

✓ PASSO 1 — Scriviamo bene il PRIMALE

$$\begin{aligned} \max \quad & x_1 + x_2 \\ \text{s.t.} \quad & x_1 + 2x_2 \leq 6 \quad (\text{vincolo 1}) \\ & 3x_1 + 2x_2 \leq 12 \quad (\text{vincolo 2}) \\ & x_1, x_2 \geq 0 \end{aligned}$$

✓ PASSO 2 — Riconosci cosa hai nel primale

- È un problema di massimo
- I vincoli sono del tipo $\leq$
- Le variabili sono x_1, x_2
- Ci sono **2 vincoli** $\Rightarrow$ il duale avrà **2 variabili**, che chiameremo y_1, y_2
- Ci sono **2 variabili** $\Rightarrow$ il duale avrà **2 vincoli**

Dal PRIMALE	Diventa nel DUALE	
$\max c^T x$	$\min b^T y$	
m vincoli $\Rightarrow m$ variabili y_i		
n variabili $\Rightarrow n$ vincoli		
A (vincoli)	A^T (coeff. dei vincoli duali)	
c (obiettivo primale)	Lato destro dei vincoli duali	
b (vincoli primale)	Obiettivo del duale	
$\leq$ nei vincoli primale	$y_i \geq 0$ nel duale	
$x_j \geq 0$	vincoli $\geq$ nel duale	



ESEMPIO COMPLETO – TUTTO INSIEME

◆ PRIMALE:

$$\begin{aligned} \max \quad & x_1 + x_2 \\ \text{s.t.} \quad & x_1 + 2x_2 \leq 6 \quad (\text{vincolo 1}) \\ & 3x_1 + 2x_2 \leq 12 \quad (\text{vincolo 2}) \\ & x_1, x_2 \geq 0 \end{aligned}$$

👉 Quindi:

- $c = (1, 1)$
- $b = (6, 12)$
- Matrice $A = \begin{bmatrix} 1 & 2 \\ 3 & 2 \end{bmatrix}$

◆ DUALE (regola per regola):

$$\begin{aligned} \min \quad & 6y_1 + 12y_2 \\ \text{s.t.} \quad & y_1 + 3y_2 \geq 1 \quad (\text{vincolo su } x_1) \\ & 2y_1 + 2y_2 \geq 1 \quad (\text{vincolo su } x_2) \\ & y_1, y_2 \geq 0 \end{aligned}$$

- Trasformare un duale in primale

✓ Ecco il DUALE di partenza

$$\begin{aligned} \min & 6y_1 + 12y_2 \\ \text{s.t. } & y_1 + 3y_2 \geq 1 \quad (\text{vincolo 1}) \\ & 2y_1 + 2y_2 \geq 1 \quad (\text{vincolo 2}) \\ & y_1, y_2 \geq 0 \end{aligned}$$

📌 STEP 1 – Capisci la struttura del duale:

- È un problema di minimo
- Ha 2 variabili: y_1, y_2
- Ha 2 vincoli $\Rightarrow$ il primale avrà 2 variabili
- Ha 2 variabili $\Rightarrow$ il primale avrà 2 vincoli

Dal problema DUALE (min)	Diventa nel problema PRIMALE (max)
n variabili $y_1, y_2, \dots, y_n$	$\rightarrow$ n vincoli nel primale
m vincoli nel duale	$\rightarrow$ m variabili $x_1, x_2, \dots, x_m$
Coefficiente obiettivo $b = (b_1, b_2, \dots)$	$\rightarrow$ termini noti del primale (lato destro dei vincoli)
Coefficienti nei vincoli del duale (matrice)	$\rightarrow$ colonne della matrice A del primale
Vincoli $\geq$ e $y \geq 0$	$\rightarrow$ vincoli $\leq$ e $x \geq 0$

📌 STEP 3 – Scriviamo la matrice A del primale

Prendiamo i coefficienti dei vincoli nel duale:

- $y_1 + 3y_2 \geq 1 \rightarrow$ colonna 1: $\begin{bmatrix} 1 \\ 3 \end{bmatrix}$
- $2y_1 + 2y_2 \geq 1 \rightarrow$ colonna 2: $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$

Quindi, la matrice A del primale è:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 2 \end{bmatrix}$$

📌 STEP 4 – Costruisci il primale

- $c = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (dai termini noti dei vincoli nel duale)
- $b = \begin{bmatrix} 6 \\ 12 \end{bmatrix}$ (dal coeff. obiettivo del duale)

◆ Quindi il primale è:

$$\begin{aligned} \max & x_1 + x_2 \\ \text{s.t. } & x_1 + 2x_2 \leq 6 \\ & 3x_1 + 2x_2 \leq 12 \\ & x_1, x_2 \geq 0 \end{aligned}$$

✓ Esattamente il problema primale di partenza!

- **Perché ci interessa il duale?**

Il problema duale ci interessa perchè ci aiuta a verificare se esistono soluzioni ottime per il primale e a certificare che la soluzione ottima è effettivamente ottima.

- **Teorema debole della dualità**

Per ogni coppia di soluzioni ammissibili del primale e del duale, il valore della funzione obiettivo del primale è minore o uguale a quello del duale:

$$c^T x \leq b^T y$$

 Esempio:

PRIMALE:

$$\begin{aligned} \max \quad & x_1 + x_2 \\ \text{s.t.} \quad & x_1 + 2x_2 \leq 6 \\ & 3x_1 + 2x_2 \leq 12 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Soluzione ammissibile: $x = (2, 2) \Rightarrow z = 2 + 2 = 4$

DUALE:

$$\begin{aligned} \min \quad & 6y_1 + 12y_2 \\ \text{s.t.} \quad & y_1 + 3y_2 \geq 1 \\ & 2y_1 + 2y_2 \geq 1 \\ & y_1, y_2 \geq 0 \end{aligned}$$

Soluzione ammissibile: $y = (0.25, 0.25) \Rightarrow z = 6(0.25) + 12(0.25) = 4.5$

👉 Qui: $c^T x = 4 \leq 4.5 = b^T y$

✅ Teorema debole della dualità confermato!

- **Teorema forte della dualità**

Se il **primale** ha una **soluzione ottima**, allora anche il **duale** ce l'ha, e i **valori** **coincidono**.

Cioè:

$$\max c^T x = \min b^T y$$

Quindi: se trovi una **soluzione ottima** nel **primale**, puoi trovare anche quella del **duale**, e **hanno lo stesso valore obiettivo**.

◆ **PRIMALE:**

$$\begin{aligned} \max \quad & x_1 + x_2 \\ \text{s.t.} \quad & x_1 + 2x_2 \leq 6 \\ & 3x_1 + 2x_2 \leq 12 \\ & x_1, x_2 \geq 0 \end{aligned}$$

✦ Soluzione ottima trovata:

- $x_1 = 3, x_2 = 1.5$
- Valore ottimo:

$$x_1 + x_2 = 3 + 1.5 = \boxed{4.5}$$

◆ **DUALE:**

$$\begin{aligned} \min \quad & 6y_1 + 12y_2 \\ \text{s.t.} \quad & y_1 + 3y_2 \geq 1 \\ & 2y_1 + 2y_2 \geq 1 \\ & y_1, y_2 \geq 0 \end{aligned}$$

✦ Soluzione ottima trovata:

- $y_1 = 0.25, y_2 = 0.25$
- Valore ottimo:

$$6(0.25) + 12(0.25) = 1.5 + 3 = \boxed{4.5}$$

🧠 **CONCLUSIONE:**

$$\boxed{c^T x = b^T y = 4.5}$$

- ✓ Il **primale** ha **soluzione ottima**
- ✓ Il **duale** ha **soluzione ottima**
- ✓ I due valori **coincidono**

- Una soluzione del duale può essere negativa?

♦ Dipende dal tipo di vincoli del primale.

Tipo vincolo nel primale	Vincolo sulla variabile duale
$\leq$	$y_i \geq 0$
$=$	$y_i \in \mathbb{R}$ (libero)
$\geq$	$y_i \leq 0$

👉 Quindi:

- ✓ Sì, una variabile del duale può essere negativa,
- ! solo se il primale ha vincoli di tipo $\geq$ o uguaglianze

✎ Esempio:

PRIMALE:

$$\begin{aligned} \max \quad & x_1 + x_2 \\ \text{s.t.} \quad & x_1 - x_2 \geq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

→ Il duale avrà una variabile $y \leq 0$

- come si calcola A_B^{-1} ?

- A è la matrice dei vincoli
- Una **base** B è un sottoinsieme di n colonne **linearmente indipendenti** di A
- A_B è la matrice formata da quelle colonne

Per esempio, se:

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 2 & 0 \end{bmatrix} \quad \text{e scegli } B = \{1, 2\} \Rightarrow A_B = \begin{bmatrix} 1 & 2 \\ 3 & 2 \end{bmatrix}$$

Se A_B è quadrata ($n \times n$) e invertibile, puoi fare:

$$\text{Se } A_B = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow A_B^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$A_B = \begin{bmatrix} 1 & 2 \\ 3 & 2 \end{bmatrix} \Rightarrow \det = 1 \times 2 - 2 \times 3 = -4 \Rightarrow A_B^{-1} = \frac{1}{-4} \begin{bmatrix} 2 & -2 \\ -3 & 1 \end{bmatrix} = \begin{bmatrix} -0.5 & 0.5 \\ 0.75 & -0.25 \end{bmatrix}$$

👉 Poi usi questo per calcolare:

- $\bar{x}_B = A_B^{-1} b_B$
- $\bar{y} = c_B A_B^{-1}$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad x = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$$Ax = \begin{bmatrix} 1 \times 5 + 2 \times 6 \\ 3 \times 5 + 4 \times 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

$$\det(A) \neq 0$$

✓ Se **determinante $\neq 0$** $\Rightarrow$ la matrice è **invertibile**

✗ Se **determinante $= 0$** $\Rightarrow$ **non invertibile** (matrice "singolare")

- **Teorema di Farkas**

In sostanza, afferma che per un dato sistema di disequazioni lineari, o si può trovare una soluzione che soddisfa tutte le disequazioni, oppure esiste una combinazione lineare delle stesse che porta ad una contraddizione. In altre parole, una delle due alternative è sempre vera, ma mai entrambe.

Dato un sistema $Ax = b$, esattamente una delle seguenti affermazioni è vera:

1. Esiste $x \geq 0$ tale che $Ax = b$ (soluzione esiste),
oppure
2. Esiste y tale che:
 - $y^T A \geq 0$
 - $y^T b < 0$

✓ COSA SIGNIFICA $y^T A$?

È una moltiplicazione di vettori/matrici, più precisamente:

$y^T A$ significa: vettore riga y trasposto moltiplicato per la matrice A .

Se:

- $y \in \mathbb{R}^m$ è un vettore colonna
- $A \in \mathbb{R}^{m \times n}$

Allora:

- $y^T \in \mathbb{R}^{1 \times m}$ è un vettore riga
- $y^T A \in \mathbb{R}^{1 \times n}$ è un nuovo vettore riga con n elementi

👉 Si ottiene moltiplicando riga per riga.

✏ Eempio:

Sia

$$y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

Allora:

$$y^T = [1 \quad 2]$$

Ora calcoliamo $y^T A$:

$$y^T A = [1 \quad 2] \cdot \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} = [1 \times 1 + 2 \times 2 \quad 1 \times 3 + 2 \times 4] = [5 \quad 11]$$

✳ Risultato: $y^T A = [5, 11]$

✓ COSA SIGNIFICA $y^T b$?

È semplicemente il **prodotto scalare** tra il vettore y e il vettore b .

Se:

- $y \in \mathbb{R}^m$
- $b \in \mathbb{R}^m$

Allora:

$$y^T b = y_1 b_1 + y_2 b_2 + \dots + y_m b_m$$

✏ Eempio:

$$y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \Rightarrow y^T b = 1 \times 3 + 2 \times 4 = 3 + 8 = \boxed{11}$$

ESEMPIO TEOREMA DI FARKAS:

ESEMPIO

Considera il sistema:

$$\begin{cases} x_1 + x_2 = 1 \\ x_1 + x_2 = 3 \\ x_1, x_2 \geq 0 \end{cases}$$

Verifica se ha soluzioni

Hai due equazioni:

- $x_1 + x_2 = 1$
- $x_1 + x_2 = 3$

✗ Sono in contraddizione $\Rightarrow$ nessuna soluzione

→ Il sistema $Ax = b, x \geq 0$ è incompatibile

Scriviamolo in forma matriciale

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

Sistema:

$$Ax = b, \quad x \geq 0$$

Teorema di Farkas: troviamo un y tale che:

$$y^T A \geq 0 \quad \text{e} \quad y^T b < 0$$

✓ Prova con $y = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

1. Calcolo $y^T A$:

$$y^T A = [1 \quad -1] \cdot \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = [1 \times 1 + (-1) \times 1, 1 \times 1 + (-1) \times 1] = [0 \quad 0] \geq 0 \quad \checkmark \text{ ok}$$

2. Calcolo $y^T b$:

$$y^T b = [1 \quad -1] \cdot \begin{bmatrix} 1 \\ 3 \end{bmatrix} = 1 \times 1 + (-1) \times 3 = 1 - 3 = -2 < 0 \quad \checkmark \text{ ok}$$

✓ Abbiamo trovato un vettore y tale che:

- $y^T A \geq 0$
- $y^T b < 0$

→ Quindi, per il Teorema di Farkas, il sistema non ha soluzione

→ $y = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ è il "certificato di infeasibilità"

Algoritmi del simplesso

- Descrivi l'algoritmo del simplesso primale (anche pseudocodice)

```

procedure (  $B, \bar{x}, \bar{y}, stato$  ) = Simplesso_Primale(  $A, b, c, B$  ) {
  for(  $stato = "iniziale"$  ; ) { // invariante:  $B$  primale ammissibile
     $\bar{x} = A_B^{-1}b_B$ ;  $\bar{y} = [\bar{y}_B, \bar{y}_N] = [cA_B^{-1}, 0]$ ;
    if(  $\bar{y}_B \geq 0$  ) then {  $stato = "ottimo"$ ; break; }
     $h = \min\{i \in B : \bar{y}_i < 0\}$ ;  $\xi = -A_B^{-1}u_{B(h)}$ ;
    if(  $A_N\xi \leq 0$  ) then {  $stato = "P.illimitato"$ ; break; }
     $\bar{\lambda} = \min\{\lambda_i = (b_i - A_i\bar{x}) / (A_i\xi), A_i\xi > 0, i \in N\}$ ;
     $k = \min\{i \in N : \lambda_i = \bar{\lambda}\}$ ;  $B = B \cup \{k\} \setminus \{h\}$ ;
  }
}

```

Procedura 3.1: Semplesso Primale

L'algoritmo riceve in input una descrizione del problema ed un base B che si assume essere *primale ammissibile*, ed iterativamente esegue le seguenti operazioni:

1. Verifica l'ottimalità della soluzione primale di base $\bar{x}$, nel qual caso termina fornendo anche la corrispondente soluzione duale $\bar{y}$, oppure individua una direzione di crescita ξ .
2. Calcola il massimo passo di spostamento lungo la direzione ξ , che può essere nullo in caso di degenerazione primale: se il passo risulta essere $+\infty$, cioè se la direzione ξ consente una crescita illimitata della funzione obiettivo, allora l'algoritmo termina avendo provato che (P) è illimitato e quindi (D) è vuoto.
3. Viene aggiornata la base: $B' = B \cup \{k\} \setminus \{h\}$, con h e k selezionati come precedentemente definito; ciò corrisponde o ad un cambiamento di vertice, ed in questo caso il valore della funzione obiettivo cresce, oppure ad un cambiamento di base degenero.

L'algoritmo termina in un numero finito di passi fornendo una base ottima B e la corrispondente coppia di soluzioni ottima, se esiste, oppure fornendo la risposta che il problema è illimitato.

Esempio 3.30. Esecuzione dell'algoritmo del simplesso primale

Consideriamo la coppia asimmetrica di problemi duali dell'Esempio 3.23 e applichiamo l'algoritmo del simplesso primale a partire dalla base primale ammissibile $B^1 = \{2, 5\}$ già analizzata in quell'esempio, a cui corrisponde il vertice $\bar{x}^1 = [4, 4]$. Come abbiamo già visto, la soluzione duale di base è $\bar{y}^1 = [0, 1, 0, 0, -5]$: quindi l'indice uscente è $h = 5$, con $B^1(h) = 2$ e la direzione di crescita è

$$\xi^1 = -A_{B^1}^{-1}u_{B^1(h)} = \begin{bmatrix} -1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

La ricerca dell'indice uscente è effettuata analizzando i vincoli non in base:

$$A_{N^1}\xi^1 = \begin{bmatrix} -2 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -3 \\ 2 \\ 3 \end{bmatrix}.$$

Essendo positive solo $A_3\xi^1$ ed $A_4\xi^1$, l'insieme $J^1 = \{i \in N^1 : A_i\xi^1 > 0\}$ degli indici candidati ad entrare in base è $\{3, 4\}$; infatti, dalla Figura 3.6 si può osservare come, partendo da $\bar{x}^1$, e muovendosi nella direzione ξ^1 lungo l'iperpiano (retta) di supporto del secondo vincolo, si incontrano gli iperpiani (rette) di supporto dei vincoli 3 e 4. Il valore negativo di $A_1\xi^1$ indica che ci si sta allontanando dall'iperpiano (retta) di supporto del vincolo 1.

Determiniamo ora il passo dello spostamento lungo ξ^1 , dato da $\bar{\lambda} = \min\{\lambda_i : i \in J^1\}$, dove λ_i è data da (3.19):

$$\lambda_3 = (14 - 8) / 3 = 2, \quad \lambda_4 = (8 - 4) / 2 = 2;$$

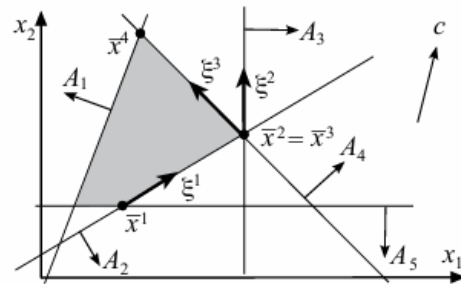
essendo $\bar{\lambda} = \lambda_3 = \lambda_4 = 2$, per la regola anticiclo di Bland si ha $k = \min\{3, 4\} = 3$. La nuova base è perciò $B^2 = B^1 \setminus \{h\} \cup \{k\} = \{2, 5\} \setminus \{5\} \cup \{3\} = \{2, 3\}$.

Effettuiamo la seconda iterazione:

$$A_{B^2} = \begin{bmatrix} 1 & -2 \\ 1 & 0 \end{bmatrix}, \quad A_{B^2}^{-1} = \begin{bmatrix} 0 & 1 \\ -1/2 & 1/2 \end{bmatrix}, \quad b_{B^2} = \begin{bmatrix} -4 \\ 8 \end{bmatrix};$$

pertanto la nuova soluzione di base primale è

$$\bar{x}^2 = A_{B^2}^{-1}b_{B^2} = \begin{bmatrix} 0 & 1 \\ -1/2 & 1/2 \end{bmatrix} \begin{bmatrix} -4 \\ 8 \end{bmatrix} = \begin{bmatrix} 8 \\ 6 \end{bmatrix}$$



mentre quella duale è

$$\bar{y}_{B^2} = cA_{B^2}^{-1} = \begin{bmatrix} 1 & 3 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ -1/2 & 1/2 \end{bmatrix} = \begin{bmatrix} -3/2 & 5/2 \end{bmatrix} \quad , \quad \bar{y}_{N^2} = 0$$

e quindi $\bar{y}^2 = [0, -3/2, 5/2, 0, 0]$. Essendoci solo $\bar{y}_2^2 < 0$, l'indice uscente è necessariamente $h = 2$. Calcoliamo la direzione di crescita e il passo lungo essa:

$$\xi^2 = \begin{bmatrix} 0 & -1 \\ 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1/2 \end{bmatrix} \quad , \quad A_{N^2}\xi^2 = \begin{bmatrix} -2 & 1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1/2 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 1/2 \\ -1/2 \end{bmatrix} \quad ,$$

$$J^2 = \{1, 4\} \quad , \quad \lambda_1 = 22 \quad , \quad \lambda_4 = 0 = \bar{\lambda} \quad , \quad k = 4 \quad .$$

$\lambda_4 = 0$ è una conseguenza del fatto che $A_4\bar{x}^2 = 14 = b_4$, cioè che la soluzione di base è primale degenera, come avevamo già mostrato nell'Esempio 3.24. Pertanto, $\bar{\lambda} = 0$ indica che la direzione di crescita ξ^2 non è una direzione ammissibile. La nuova base è $B^3 = B^2 \setminus \{2\} \cup \{4\} = \{3, 4\}$ e la corrispondente soluzione primale di base è

$$A_{B^3} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \quad , \quad A_{B^3}^{-1} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \quad , \quad b_{B^3} = \begin{bmatrix} 8 \\ 14 \end{bmatrix} \quad , \quad \bar{x}^3 = \begin{bmatrix} 8 \\ 6 \end{bmatrix} \quad .$$

Abbiamo effettuato un cambio di base degenera, in quanto la soluzione primale individuata coincide con la precedente; la nuova soluzione di base duale, come visto nell'Esempio 3.24, è diversa dalla precedente, infatti:

$$\bar{y}_{B^3} = \begin{bmatrix} 1 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} -2 & 3 \end{bmatrix} \quad , \quad \bar{y}_{N^3} = 0 \quad , \quad \bar{y}^3 = \begin{bmatrix} 0 & 0 & -2 & 3 & 0 \end{bmatrix} \quad .$$

L'indice uscente è quindi $h = 3$, e

$$\xi^3 = \begin{bmatrix} -1 & 0 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \quad , \quad A_{N^3}\xi^3 = \begin{bmatrix} -2 & 1 \\ 1 & -2 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ -3 \\ -1 \end{bmatrix} \quad ;$$

si ha pertanto $J^3 = \{1\}$ e $\bar{\lambda} = \lambda_1 = 11/3$, l'indice entrante è $k = 1$ e la nuova base è $B^4 = B^3 \setminus \{3\} \cup \{1\} = \{1, 4\}$. Poiché

$$A_{B^4} = \begin{bmatrix} -2 & 1 \\ 1 & 1 \end{bmatrix} \quad , \quad A_{B^4}^{-1} = \begin{bmatrix} -1/3 & 1/3 \\ 1/3 & 2/3 \end{bmatrix} \quad , \quad b_{B^4} = \begin{bmatrix} 1 \\ 14 \end{bmatrix} \quad , \quad \bar{x}^4 = \begin{bmatrix} 13/3 \\ 29/3 \end{bmatrix}$$

$$\bar{y}_{B^4} = \begin{bmatrix} 2/3 & 7/3 \end{bmatrix} \quad , \quad \bar{y}_{N^4} = 0 \quad , \quad \bar{y}^4 = \begin{bmatrix} 2/3 & 0 & 0 & 7/3 & 0 \end{bmatrix} \geq 0$$

la soluzione duale è ammissibile e quindi l'algoritmo termina: $\bar{x}^4, \bar{y}^4$ costituiscono la coppia di soluzioni ottime associate alla base ottima $B = \{1, 4\}$. La sequenza di soluzioni primali e di direzioni di crescita è mostrata nella figura in alto.

L'algoritmo del simplesso primale, descritto nel precedente paragrafo, genera una sequenza di coppie di soluzioni di base primali ammissibili e duali non ammissibili (tranne al più l'ultima): se la soluzione di base duale risulta ammissibile allora l'algoritmo termina, avendo determinato un vertice ottimo.

- **Descrivi l'algoritmo del simplesso duale (anche pseudocodice)**

◆ STEP 0 — Forma standard del primale

$$\max c^T x \quad \text{s.t. } Ax = b, \quad x \geq 0$$

◆ STEP 1 — Parti da una base non ammissibile

Hai:

- A_B : matrice di base
- b_B : lato destro $\rightarrow$ calcola $\bar{x}_B = A_B^{-1}b$

⚠ Se $\bar{x}_B \not\geq 0$, la base **non** è ammissibile per il primale

✓ Ma calcoli $\bar{y} = c_B A_B^{-1}$ e ottieni una soluzione **dualmente ammissibile**

◆ STEP 2 — Scegli la variabile uscente

- Trova un indice $h \in B$ per cui $\bar{x}_h < 0$
- Vuol dire che **quel vincolo va sistemato**, cioè quella variabile deve uscire dalla base

◆ STEP 3 — Costruisci la direzione ξ

- Calcola $\xi = A_B^{-1}A_k$, per ogni variabile $k \notin B$

◆ STEP 4 — Calcola i coefficienti duali ridotti (gli "scarti")

Per ogni $k \notin B$, calcola:

$$\bar{c}_k = c_k - \bar{y}A_k$$

Se esiste almeno un $\bar{c}_k > 0$ con $\xi_h < 0$, allora puoi fare un passo.

◆ STEP 5 — Calcola il passo massimo (λ)

$$\lambda_k = \frac{\bar{c}_k}{-\xi_h} \quad (\text{solo per quelli con } \xi_h < 0)$$

Scegli il **minimo** $\lambda_k \rightarrow$ corrisponde all'indice k :

✓ Quella variabile **entra in base**

◆ STEP 6 — Fai il pivot

- Esce x_h dalla base
- Entra x_k
- Aggiorna la base B
- Torna al passo 1

← **Si ferma quando:**

- Tutti i $\bar{x}_B \geq 0 \Rightarrow$ la base è anche **ammissibile per il primale**
- E poiché $\bar{y}$ era già ammissibile per il duale $\Rightarrow$ sei **ottimo**

Esempio 3.34. Applicazione del simplesso duale per via algebrica

Si consideri la coppia asimmetrica di problemi duali

$$(P) \quad \begin{array}{rcl} \max & x_1 & \\ & x_1 + 2x_2 \leq 6 \\ & x_1 - 2x_2 \leq 6 \\ & 2x_1 + x_2 \leq 4 \\ & 2x_1 - x_2 \leq 4 \\ & -x_1 \leq 0 \end{array} \quad (D) \quad \begin{array}{rcl} \min & 6y_1 + 6y_2 + 4y_3 + 4y_4 & \\ & y_1 + y_2 + 2y_3 + 2y_4 - y_5 = 1 \\ & 2y_1 - 2y_2 + y_3 - y_4 = 0 \\ & y_1, y_2, y_3, y_4, y_5 \geq 0 \end{array}$$

e la base $B^1 = \{1, 2\}$. Essendo

$$A_{B^1} = \begin{bmatrix} 1 & 2 \\ 1 & -2 \end{bmatrix}, \quad A_{B^1}^{-1} = \begin{bmatrix} 1/2 & 1/2 \\ 1/4 & -1/4 \end{bmatrix}, \quad b_{B^1} = \begin{bmatrix} 6 \\ 6 \end{bmatrix}$$

la corrispondente soluzione di base primale è

$$\bar{x}^1 = A_{B^1}^{-1} b_{B^1} = \begin{bmatrix} 1/2 & 1/2 \\ 1/4 & -1/4 \end{bmatrix} \begin{bmatrix} 6 \\ 6 \end{bmatrix} = \begin{bmatrix} 6 \\ 0 \end{bmatrix},$$

che risulta essere non ammissibile; infatti

$$A_{N^1} \bar{x}^1 = \begin{bmatrix} 2 & 1 \\ 2 & -1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 6 \\ 0 \end{bmatrix} = \begin{bmatrix} 12 \\ 12 \\ -6 \end{bmatrix} \not\leq \begin{bmatrix} 4 \\ 4 \\ 0 \end{bmatrix} = b_{N^1}.$$

```

procedure (  $B, \bar{x}, \bar{y}, stato$  ) = Simplesso_Duale(  $A, b, c, B$  ) {
  for(  $stato = "in"; ;$  ) { // invariante:  $B$  duale ammissibile
     $\bar{x} = A_B^{-1} b_B; \bar{y} = [\bar{y}_B, \bar{y}_N] = [c A_B^{-1}, 0];$ 
    if(  $A_N \bar{x} \leq b_N$  ) then {  $stato = "ottimo";$  break; }
     $k = \min\{i \in N : A_i \bar{x} > b_i\}; \eta_B = A_k A_B^{-1};$ 
    if(  $\eta_B \leq 0$  ) then {  $stato = "P.vuoto";$  break; }
     $\bar{\theta} = \min\{\theta_i = \bar{y}_i / \eta_i : \eta_i > 0, i \in B\};$ 
     $h = \min\{i \in B : \theta_i = \bar{\theta}\}; B = B \cup \{k\} \setminus \{h\};$ 
  }
}

```

Procedura 3.2: Semplesso Duale

Viceversa, c può essere espresso come combinazione lineare a coefficienti non negativi di A_1 ed A_2 , e quindi la base B^1 è duale ammissibile; infatti

$$\bar{y}_{B^1} = c A_{B^1}^{-1} = [1, 0] \begin{bmatrix} 1/2 & 1/2 \\ 1/4 & -1/4 \end{bmatrix} = [1/2, 1/2] \quad , \quad \bar{y}^1 = [1/2, 1/2, 0, 0, 0] \geq 0.$$

Possiamo quindi applicare l'algoritmo del simplesso duale a partire dalla base B^1 . L'indice entrante è

$$k = \min\{i \in N^1 : A_i \bar{x}^1 > b_i\} = \min\{3, 4\} = 3$$

e quindi

$$\eta_{B^1} = A_3 A_{B^1}^{-1} = [2, 1] \begin{bmatrix} 1/2 & 1/2 \\ 1/4 & -1/4 \end{bmatrix} = [5/4, 3/4]$$

$$\bar{\theta} = \min\left\{\frac{\bar{y}_1}{\eta_1}, \frac{\bar{y}_2}{\eta_2}\right\} = \min\left\{\frac{1/2}{5/4}, \frac{1/2}{3/4}\right\} = \min\left\{\frac{2}{5}, \frac{2}{3}\right\} = \frac{2}{5};$$

pertanto l'indice uscente è $h = 1$. La nuova base è $B^2 = B^1 \setminus \{h\} \cup \{k\} = \{1, 2\} \setminus \{1\} \cup \{3\} = \{3, 2\}$. Effettuiamo quindi la seconda iterazione, dove

$$A_{B^2} = \begin{bmatrix} 2 & 1 \\ 1 & -2 \end{bmatrix}, \quad A_{B^2}^{-1} = \begin{bmatrix} 2/5 & 1/5 \\ 1/5 & -2/5 \end{bmatrix}, \quad b_{B^2} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}, \quad \bar{x}^2 = \begin{bmatrix} 2/5 & 1/5 \\ 1/5 & -2/5 \end{bmatrix} \begin{bmatrix} 4 \\ 6 \end{bmatrix} = \begin{bmatrix} 14/5 \\ -8/5 \end{bmatrix}$$

$$\bar{y}_{B^2} = [1, 0] \begin{bmatrix} 2/5 & 1/5 \\ 1/5 & -2/5 \end{bmatrix} = [2/5, 1/5] \quad , \quad \bar{y}^2 = [0, 1/5, 2/5, 0, 0] \quad .$$

La soluzione primale $\bar{x}^2$ non è ammissibile in quanto

$$A_{N^2} \bar{x}^2 = \begin{bmatrix} 2/5 & 1/5 \\ 1/5 & -2/5 \end{bmatrix} \begin{bmatrix} 14/5 \\ -8/5 \end{bmatrix} = \begin{bmatrix} -2/5 \\ 36/5 \\ -14/5 \end{bmatrix} \not\leq \begin{bmatrix} 6 \\ 4 \\ 0 \end{bmatrix} = b_{N^2}$$

e quindi l'indice entrante è $k = 4$ (quello dell'unico vincolo violato); pertanto

$$\eta_{B^2} = [2, -1] \begin{bmatrix} 2/5 & 1/5 \\ 1/5 & -2/5 \end{bmatrix} = [3/5, 4/5]$$

$$\bar{\theta} = \min\left\{\frac{\bar{y}_3}{\eta_3}, \frac{\bar{y}_2}{\eta_2}\right\} = \min\left\{\frac{2/5}{3/5}, \frac{1/5}{4/5}\right\} = \min\left\{\frac{2}{3}, \frac{1}{4}\right\} = \frac{1}{4},$$

da cui l'indice uscente è $h = 2$. La nuova base è perciò $B_3 = \{3, 2\} \setminus \{2\} \cup \{4\} = \{3, 4\}$. Nella terza iterazione

$$A_{B^3} = \begin{bmatrix} 2 & 1 \\ 2 & -1 \end{bmatrix}, \quad A_{B^3}^{-1} = \begin{bmatrix} 1/4 & 1/4 \\ 1/2 & -1/2 \end{bmatrix}, \quad b_{B^3} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}, \quad \bar{x}^3 = \begin{bmatrix} 1/4 & 1/4 \\ 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 4 \\ 4 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$\bar{y}_{B^3} = [1, 0] \begin{bmatrix} 1/4 & 1/4 \\ 1/2 & -1/2 \end{bmatrix} = [1/4, 1/4] \quad , \quad \bar{y}^3 = [0, 0, 1/4, 1/4, 0] \quad .$$

Stavolta la soluzione primale $\bar{x}^3$ è ammissibile, in quanto

$$A_{N^3} \bar{x}^3 = \begin{bmatrix} 1 & 2 \\ 1 & -2 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ -2 \end{bmatrix} \leq \begin{bmatrix} 6 \\ 6 \\ 0 \end{bmatrix} = b_{N^3}$$

e quindi l'algoritmo termina: $(\bar{x}^3, \bar{y}^3)$ è la coppia di soluzioni ottime associate alla base ottima $B = \{3, 4\}$.

4. Altro / Domande miste o generali

• Ford Fulkerson, SPT, SPT-S e SPT-L: cosa fanno?

- **SPT.L.Queue** può essere utilizzata per controllare se un grafo orientato possiede cicli negativi contando il numero di estrazioni da Q di ciascun nodo: appena un nodo i viene estratto per l' n -esima volta, si può affermare con certezza che quel nodo appartiene ad un ciclo negativo. In effetti, si può mostrare che un ciclo negativo può essere effettivamente determinato percorrendo all'indietro, a partire dal nodo i trovato, il vettore di predecessori $p[\cdot]$ fino al momento in cui non si incontra nuovamente i .

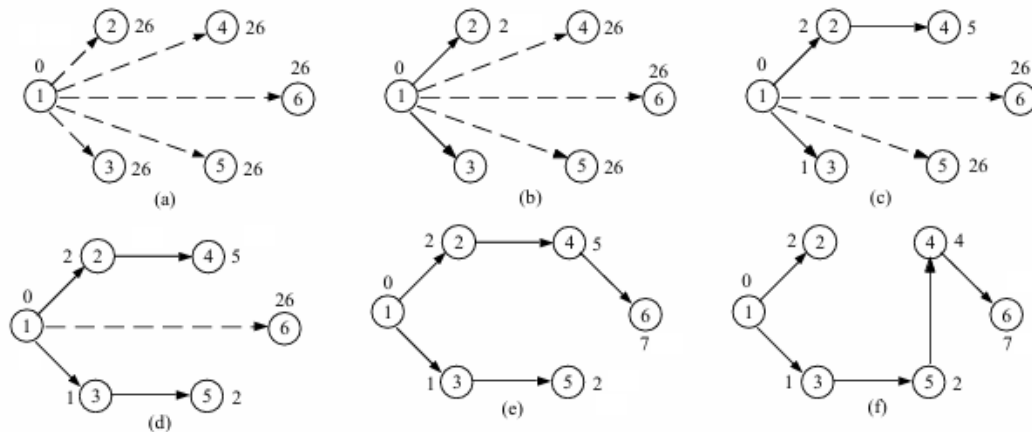
Indichiamo nel seguito con **SPT.L** le versioni di **SPT** nelle quali l'insieme Q è implementato come lista. La lista può essere realizzata in diversi modi (lista a puntatori, vettore di puntatori, lineare o circolare, semplice o doppia, ecc.), ed è sempre possibile fare in modo che le operazioni elementari ed il controllo di appartenenza di un elemento alla lista abbiano complessità costante, $O(1)$. La complessità di **SPT.L**, anche nel caso di costi negativi, dipende quindi linearmente dal numero di controlli delle condizioni di Bellman sugli archi uscenti dai nodi estratti da Q .

Fila

Esaminiamo ora l'algoritmo che si ottiene realizzando la lista Q come *fila* (*queue*), conosciuto in letteratura come *algoritmo di Bellman*: l'inserzione dei nodi avviene in coda e la rimozione dalla testa (regola FIFO).

Esempio 2.8. Esecuzione di SPT.L.Queue

Si vuole determinare l'albero dei cammini minimi di radice $r = 1$ sul grafo in Figura 2.8(a) con **SPT.L** e Q implementata come una *fila*. Gli alberi che vengono man mano costruiti sono indicati nella figura seguente. La simbologia nelle figure e nella tabella seguente coincide con quella utilizzata nell'Esempio 2.7



Iter.	Q	u	Archi esaminati	Etichette modificate	Predecessori modificati	Albero
1	{1}	1	(1, 2), (1, 3)	d_2, d_3	p_2, p_3	(b)
2	{2, 3}	2	(2, 3), (2, 4)	d_4	p_4	(c)
3	{3, 4}	3	(3, 5)	d_5	p_5	(d)
4	{4, 5}	4	(4, 6)	d_6	p_6	(e)
5	{5, 6}	5	(5, 4), (5, 6)	d_4	p_4	(f)
6	{6, 4}	6				(f)
7	{4}	4	(4, 6)	d_6	p_6	(g)
8	{6}	6				(g)

• Algoritmo di Prim?

🌀 COSA SERVE L'ALGORITMO DI PRIM?

Serve per trovare l'albero di copertura minimo (in inglese: Minimum Spanning Tree, MST) in un grafo connesso, pesato e non orientato.

🌀 Obiettivo:

Collegare tutti i nodi con un sottoinsieme di archi che:

- ✓ non contiene cicli
- ✓ ha peso totale minimo

📖 PASSI DELL'ALGORITMO DI PRIM (versione semplice)

1. Scegli un nodo iniziale qualsiasi (es. il nodo 0).
Mettilo nell'insieme dei nodi visitati (chiamiamolo T).
2. Guarda tutti gli archi che collegano nodi dentro T a nodi fuori da T
3. Scegli l'arco con peso minimo tra quelli che collegano T a un nodo fuori
4. Aggiungi all'albero:
 - quell'arco scelto
 - e il nodo nuovo a cui punta
5. Ripeti i passi 2-4 finché tutti i nodi sono stati aggiunti a T .
6. Alla fine avrai un albero con $n-1$ archi e peso totale minimo.

```

procedure  $p = \text{Prim}(G, c, r)$  {
  foreach ( $i \in V$ ) do {  $p[i] = r; d[i] = M;$  }
   $d[r] = -M; Q = \{r\};$ 
  do { seleziona  $u$  in  $Q$  tale che  $d[u] = \min\{d[j] : j \in Q\};$ 
     $d[u] = -M; Q = Q \setminus \{u\};$ 
    foreach ( $\{u, v\} \in S(u)$ ) do
      if ( $c_{uv} < d[v]$ ) then {  $d[v] = c_{uv}; p[v] = u;$ 
                           if ( $v \notin Q$ ) then  $Q = Q \cup \{v\};$ 
                           }
    } while ( $Q \neq \emptyset$ );
}
  
```

Procedura 2.5: Algoritmo di **Prim**

In Figura 2.13 sono rappresentate le soluzioni al termine delle iterazioni dell'algoritmo di **Prim** applicato al grafo dell'Esempio 2.11. I lati evidenziati sono quelli inseriti in S , e sono incidenti in vertici i tali che $p[i]$ è fissato e $d[i] = -M = -31$, mentre i lati tratteggiati sono quelli $\{p[v], v\}$ candidati ad essere inseriti in S , per ogni $v \in Q$ (accanto al nodo è riportata la sua etichetta). La linea tratteggiata indica il taglio (V', V'') . I vertici evidenziati sono quelli di etichetta minima che verranno estratti da Q all'iterazione successiva. La sequenza dei valori delle etichette dei nodi rimossi da Q è 0, 5, 3, 2, 12, 4, 16. Ad ogni iterazione viene riportato il costo $c(S)$ della soluzione corrente; al termine $c(S) = 42$, cioè la somma delle etichette dei nodi al momento della loro estrazione da Q ; infatti, esse rappresentano i costi dei lati che formano l'albero.

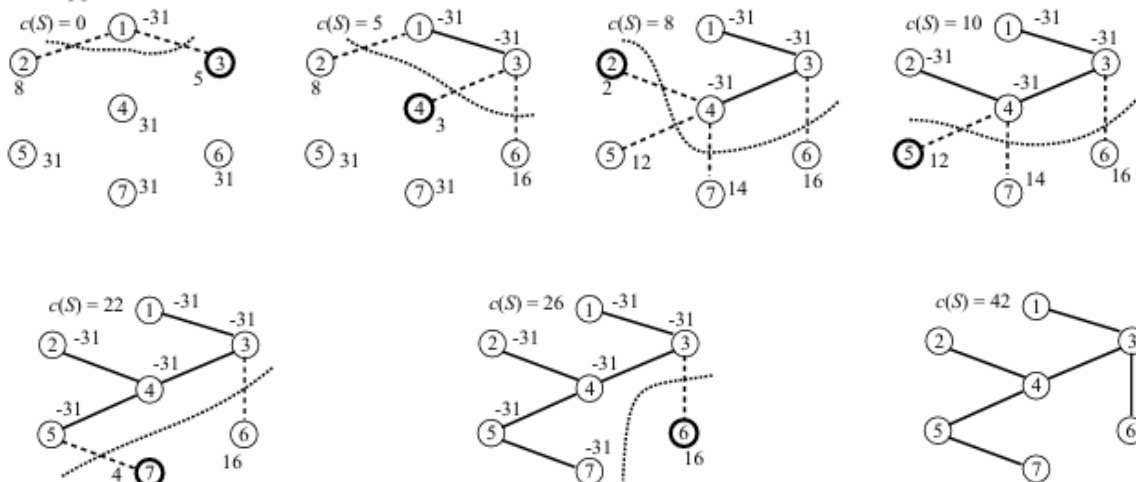


Figura 2.13: Passi effettuati dall'algoritmo di **Prim**

• Algoritmo di Kruskal?

Trovare l'albero di copertura minimo (Minimum Spanning Tree, MST) in un grafo non orientato con pesi positivi, proprio come Prim.

Ma Kruskal funziona in modo diverso:

➡ Ordina gli archi, poi li aggiunge uno alla volta, evitando di creare cicli.

📖 PASSI DELL'ALGORITMO DI KRUSKAL

1. Metti ogni nodo in un proprio "insieme" (puoi usare la struttura Union-Find)
2. Ordina tutti gli archi per peso crescente
3. Per ogni arco (u, v) nell'ordine:
 - Se u e v stanno in insiemi diversi $\rightarrow$ aggiungi l'arco all'albero
 - Unisci gli insiemi (Union)
 - Se u e v sono già connessi $\rightarrow$ salta l'arco (creerebbe un ciclo)
4. Ripeti finché hai $n-1$ archi nell'albero

```

procedure  $S = \text{Kruskal}(G, c)$  {
   $S = \emptyset$ ;  $R = \emptyset$ ;  $X = \text{Sort}(A)$ ;
  do { estrai da  $X$  il primo lato  $\{u, v\}$ ;
    if ( $\text{Component}(E, u, v)$ )
      then  $R = R \cup \{\{u, v\}\}$ ; /* cancellazione */
    else  $S = S \cup \{\{u, v\}\}$ ; /* inserzione */
  } while ( $|S| < n - 1$ );
}
  
```

Procedura 2.4: Algoritmo di Kruskal per il problema (MST)

Esempio 2.11. Esecuzione dell'algoritmo di Kruskal

Applichiamo ad l'algoritmo di Kruskal al grafo in figura qui accanto; in Figura 2.11 sono riportati i passi effettuati, riportando i lati inseriti in S ed il costo $c(S)$. L'ordinamento iniziale fornisce $X = \{\{2, 4\}, \{3, 4\}, \{5, 7\}, \{1, 3\}, \{1, 2\}, \{2, 3\}, \{4, 5\}, \{4, 7\}, \{3, 6\}, \{2, 5\}, \{6, 7\}, \{4, 6\}\}$. Si noti che il quinto lato inserito, $\{4, 5\}$, ha costo $c_{45} = 12$; tale inserimento avviene dopo la cancellazione dei lati $\{1, 2\}$ e $\{2, 3\}$ in quanto $\{1, 2\}$ forma ciclo con $\{2, 4\}$, $\{3, 4\}$ e $\{1, 3\}$, mentre $\{2, 3\}$ forma ciclo con $\{2, 4\}$ e $\{3, 4\}$. Analogamente, prima dell'inserzione di $\{3, 6\}$ vi è stata la cancellazione di $\{4, 7\}$ in quanto forma ciclo con $\{5, 7\}$ e $\{4, 5\}$. Con l'inserzione di $\{3, 6\}$ il grafo parziale definito da S diviene connesso ($|S| = 6$), e quindi un albero di copertura di costo minimo; gli ultimi tre archi nell'insieme X non vengono esaminati.

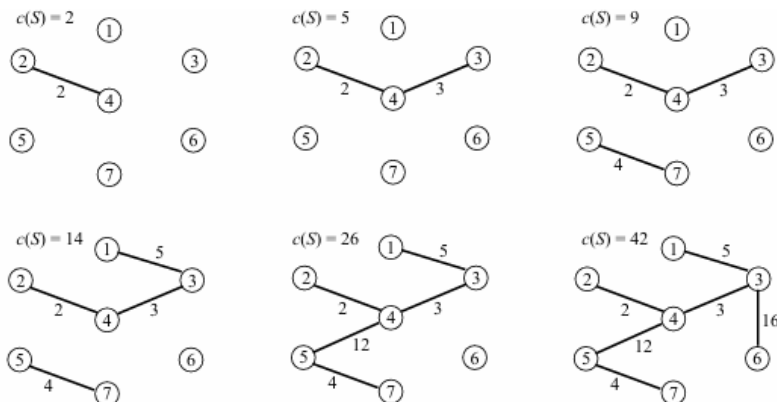
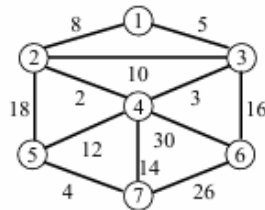


Figura 2.11: Passi effettuati dall'algoritmo di Kruskal

- **Carica fissa?**

La **carica fissa** è un costo che paghi solo se usi un arco (o una risorsa), indipendentemente da quanto flusso ci passi sopra.

 Esempio concreto:

- Hai un camion che può trasportare fino a 100 scatole da un magazzino a un negozio.
- Se lo usi anche solo per **1 scatola**, devi **comunque pagare 100€** per attivarlo (carica fissa).
- Poi magari paghi **2€/scatola** (costo variabile).

 Allora:

- **Carica fissa** = 100€ (costo che scatta appena usi l'arco)
- **Costo variabile** = 2€/scatola (dipende da quanto flusso ci metti)

 Formalmente (in un modello di flusso):

Se vuoi modellare un arco con **costo fisso** f_{ij} e **costo per unità di flusso** c_{ij} :

- Introduci una **variabile binaria** y_{ij} che vale 1 se usi l'arco (i, j) , 0 altrimenti.
- La funzione costo dell'arco sarà:

$$\text{costo}_{ij} = f_{ij} \cdot y_{ij} + c_{ij} \cdot x_{ij}$$

$$\text{con } x_{ij} \leq u_{ij} \cdot y_{ij}$$

 Questo vincolo garantisce che se non attivi l'arco ($y_{ij} = 0$), allora $x_{ij} = 0 \rightarrow$ cioè non puoi mandare flusso.